

**Last week: Local image
features**

Features of interest

Int.J. Comput. Vis. (2011) 94:154–174
DOI 10.1007/s11263-010-0340-z

Coding Images with Local Features

Timo Dicksehl · Falko Schindler · Wolfgang Förstner

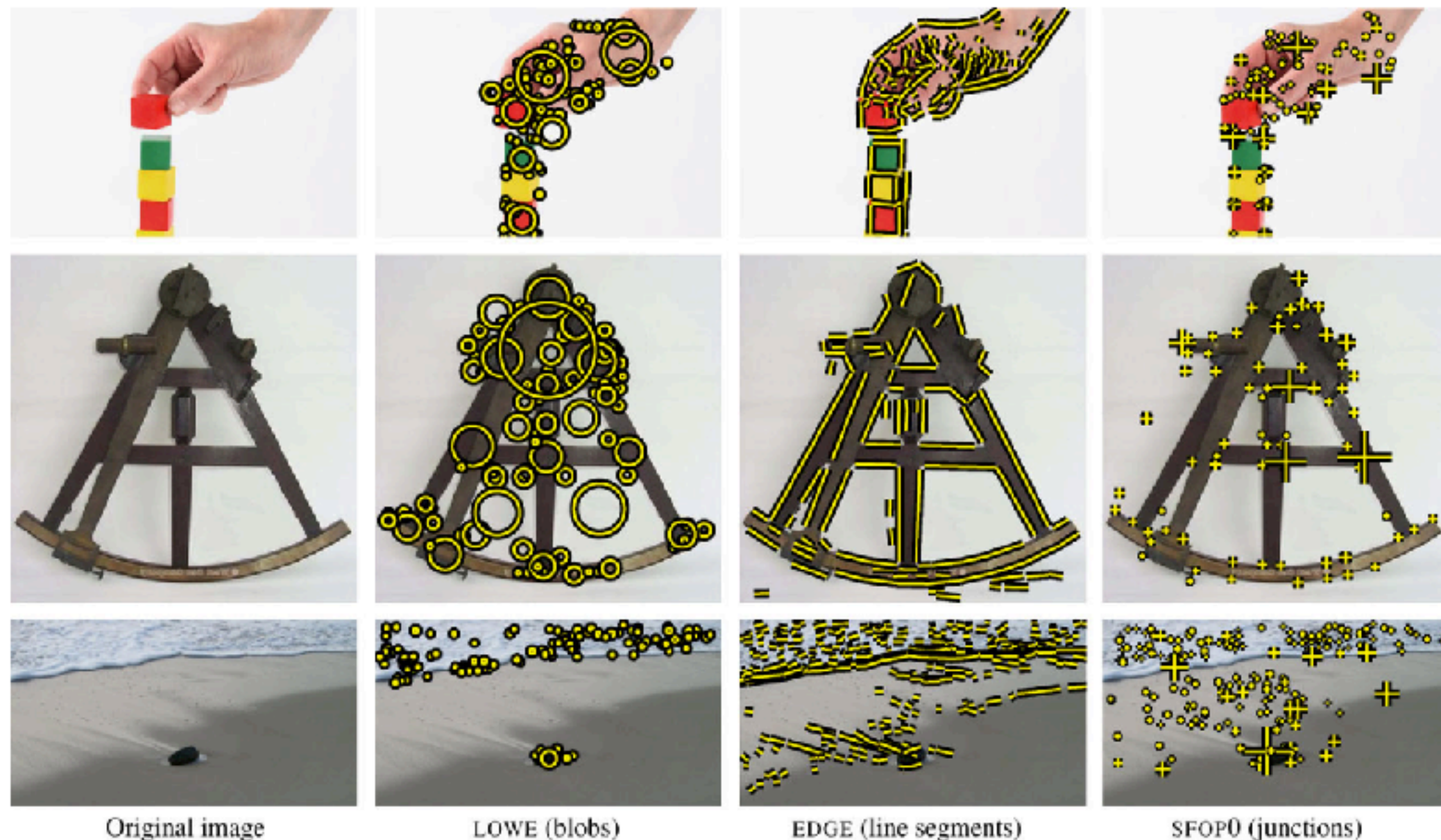
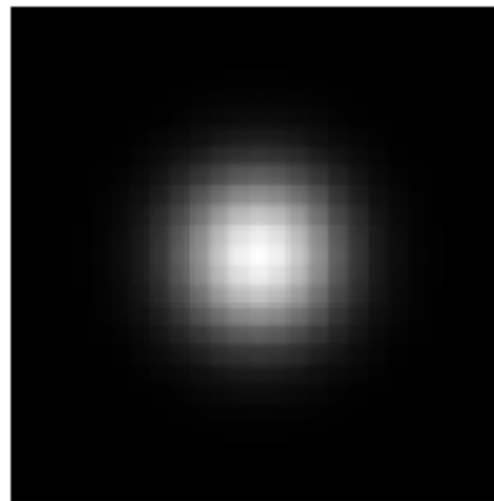
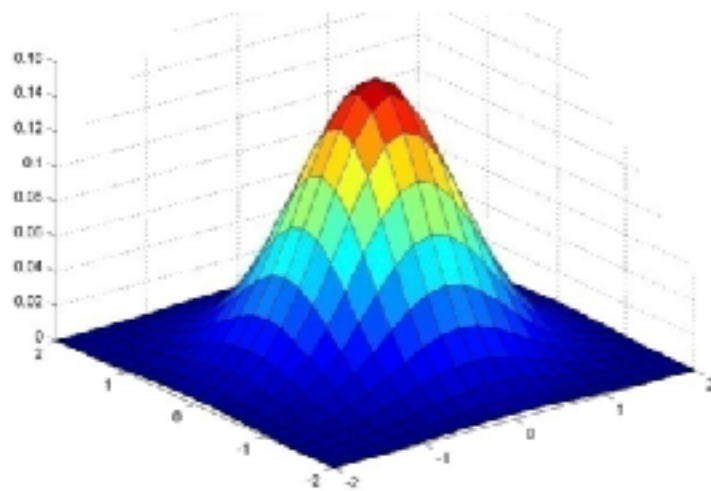


Fig. 1 Sets of extracted features on three example images. The feature detectors here are substantially different: LOWE (Lowe 2004) fires at dark and bright blobs, EDGE (Förstner 1994) yields straight edge

segments, and SFOP0 (Förstner et al. 2009, using $\alpha = 0$ in (7)) extracts junctions

Important filter: Gaussian

- Weight contributions of neighboring pixels by nearness



0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

The Laplacian filter

- The Laplace operator is defined as the sum of all 2nd derivatives

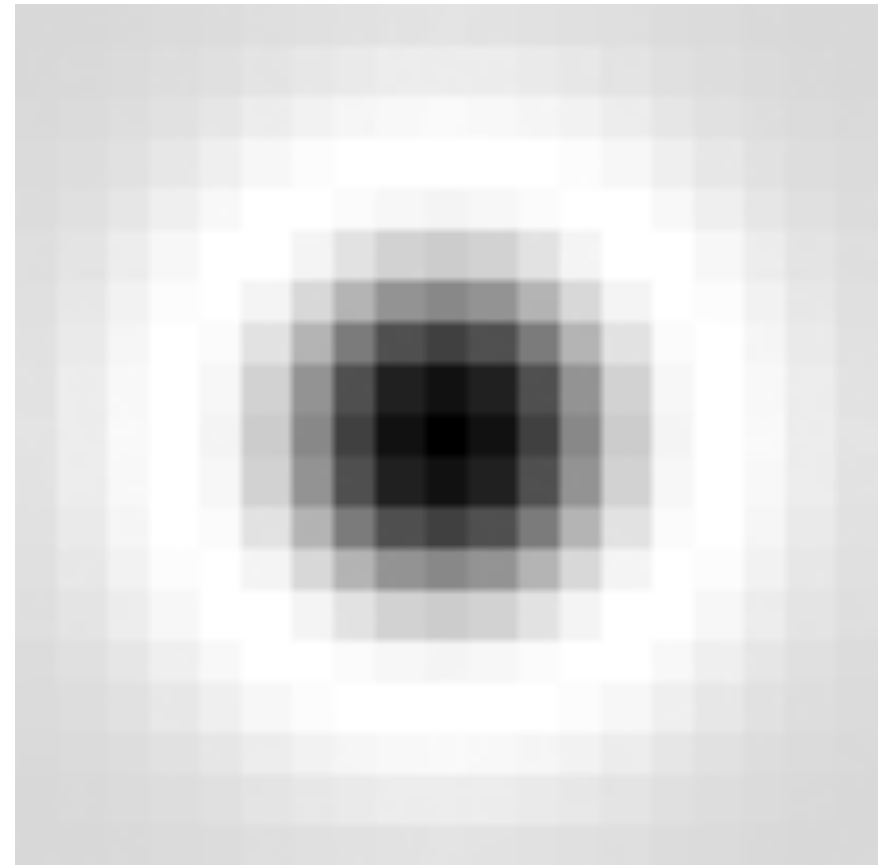
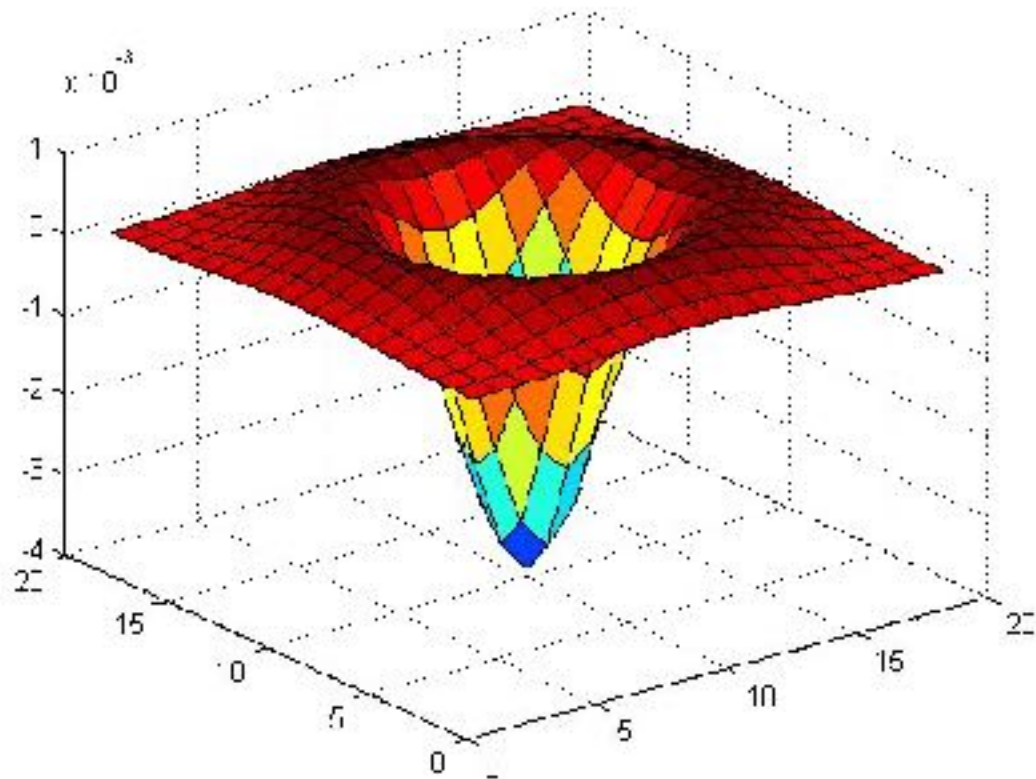
$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

- The corresponding discrete convolution filter can be modeled in different ways:

$$\mathbf{D}_{xy}^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

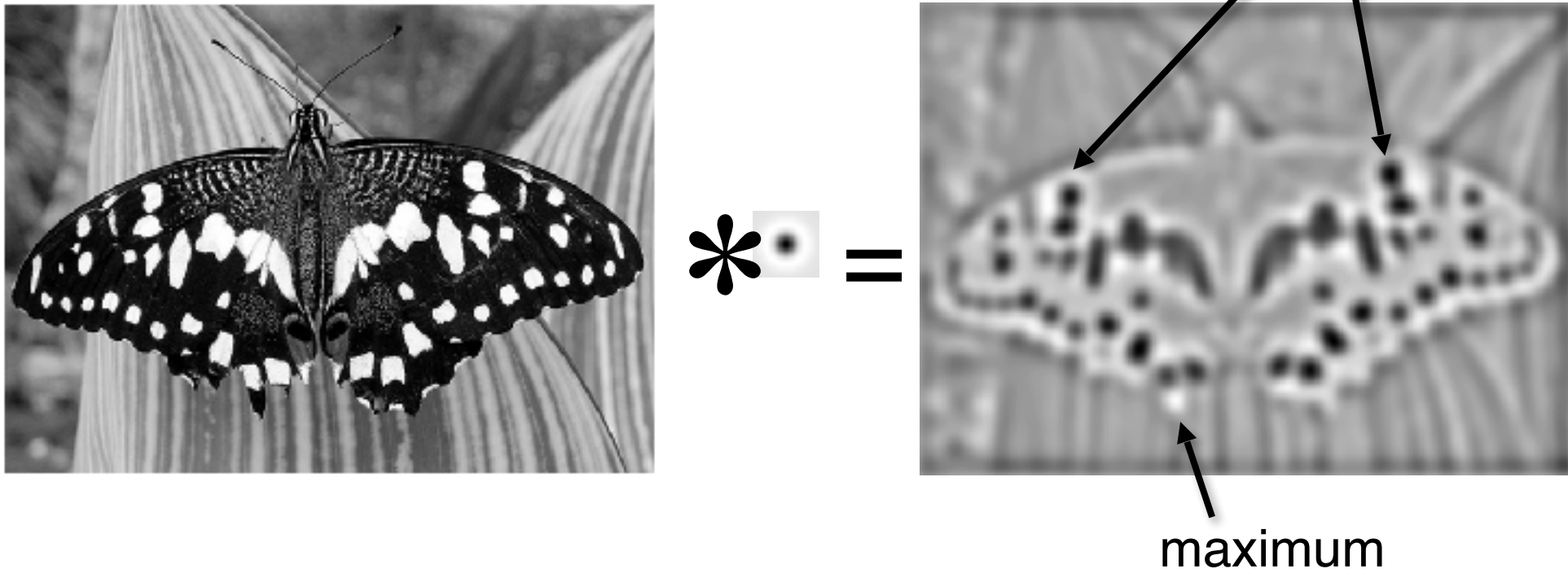
$$\mathbf{D}_{xy}^2 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Laplacian of Gaussian (LoG)



Laplacian of Gaussian

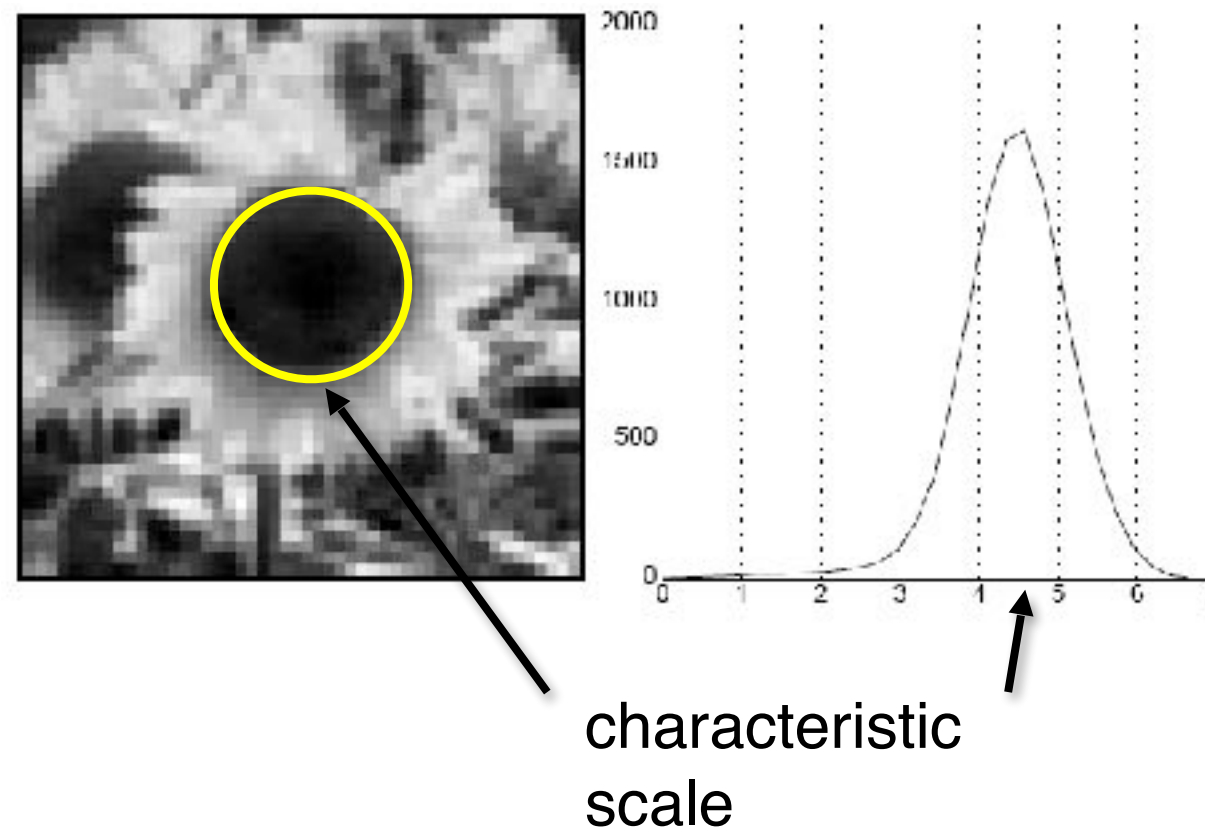
- “Blob” detector



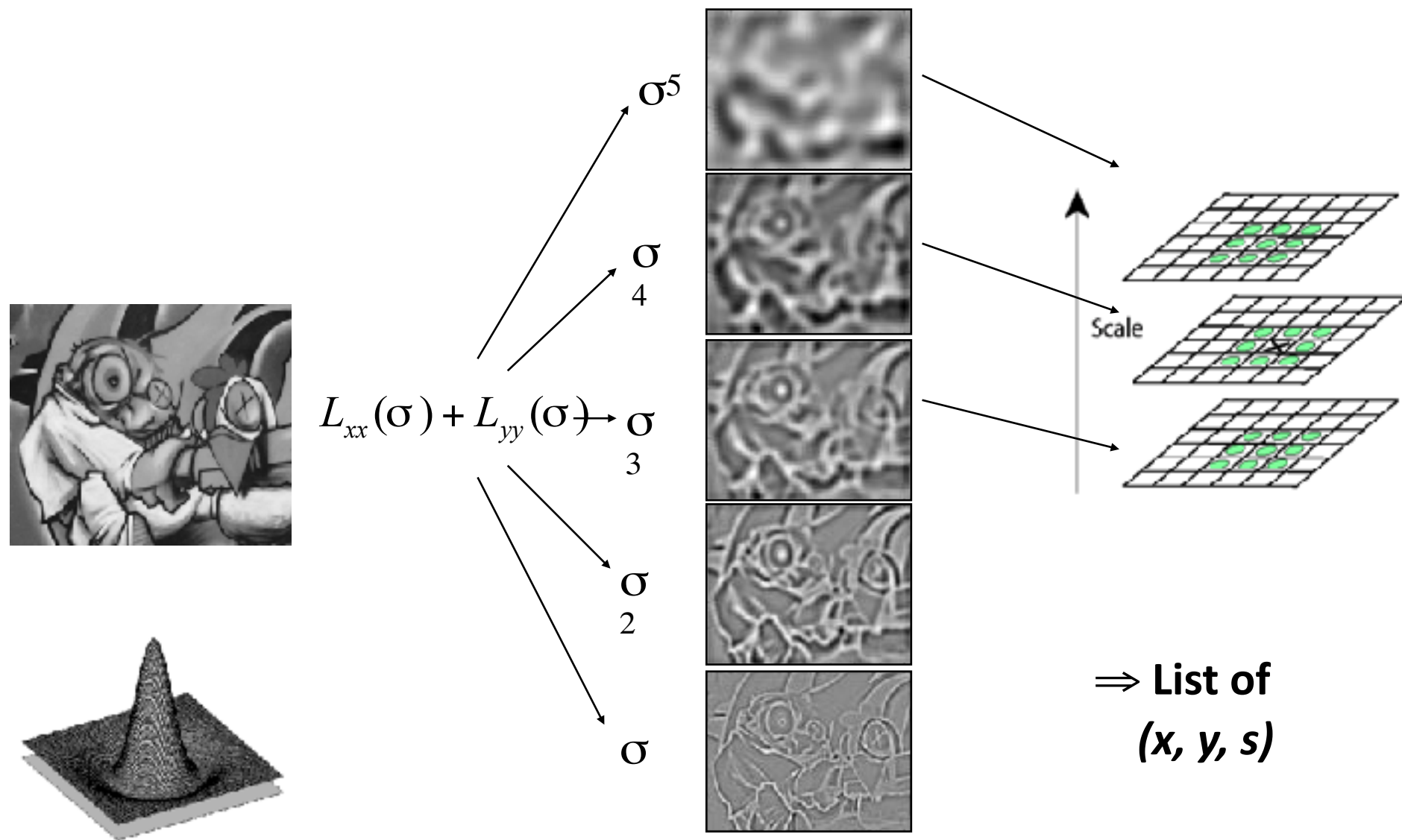
- Find maxima *and minima* of LoG operator in space and scale

Characteristic scale

- We define the characteristic scale as the scale that produces peak of Laplacian response



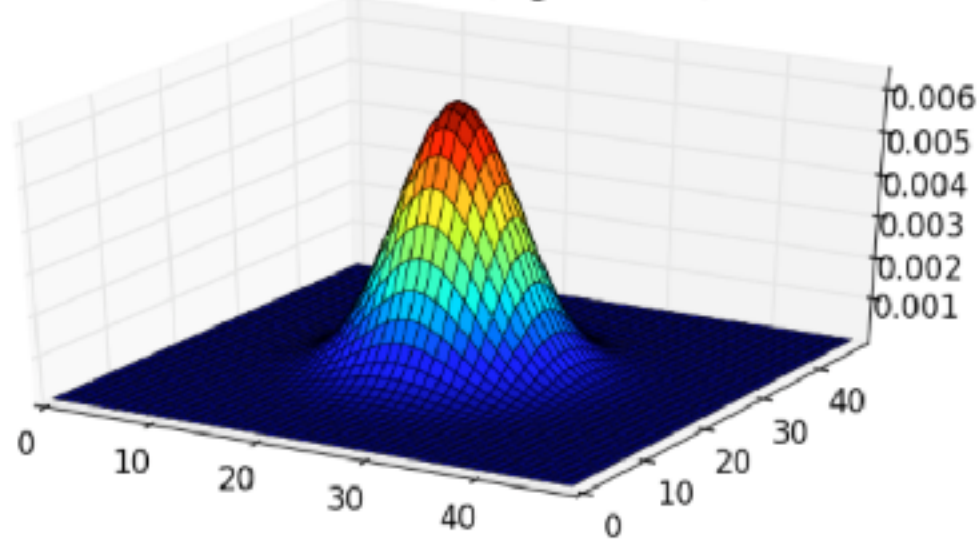
Find local maxima in 3D position-scale space



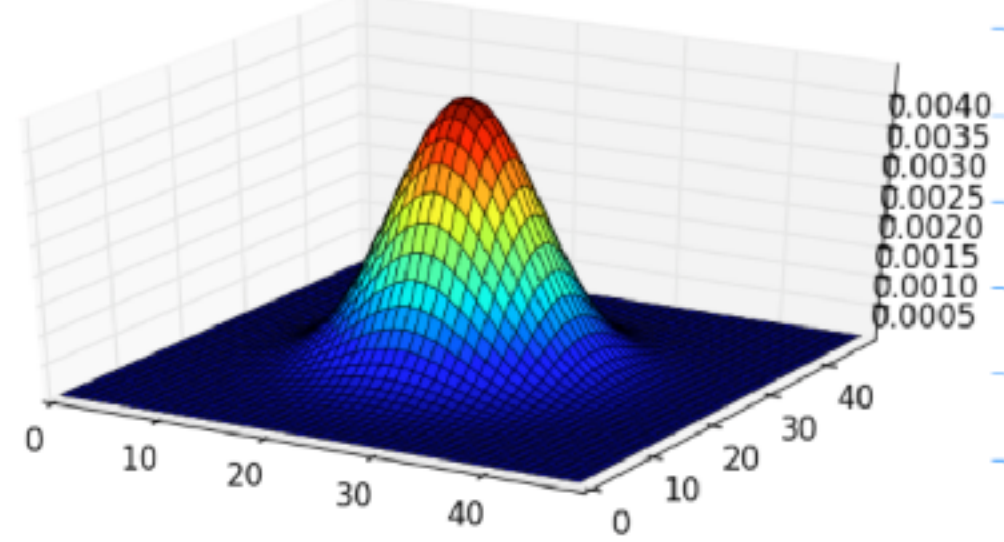
K. Grauman, B. Leibe

Difference of Gaussians (DoG)

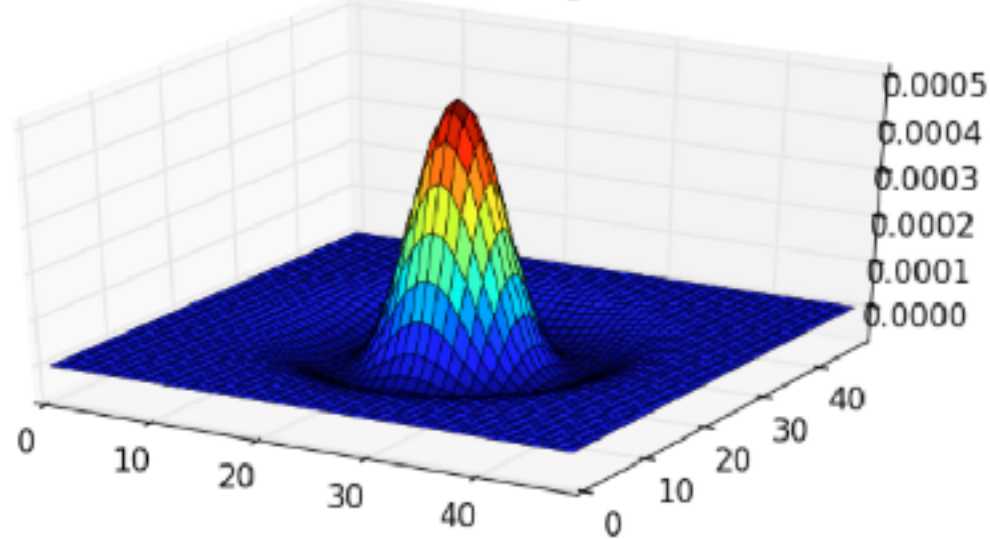
Gaussian Kernel (sigma=5.0)



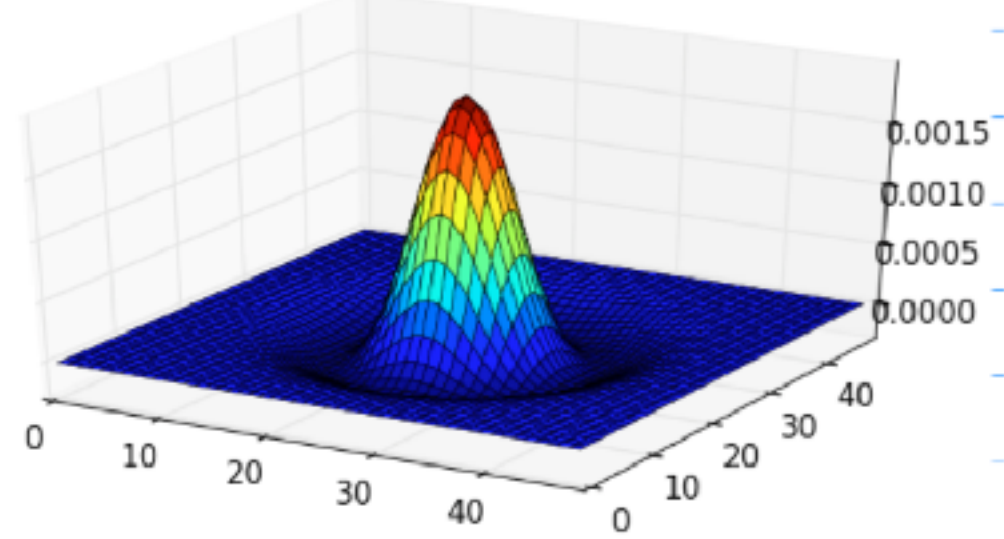
Gaussian Kernel (sigma=6.0)



Inverted LoG Kernel (sigma=5.0)



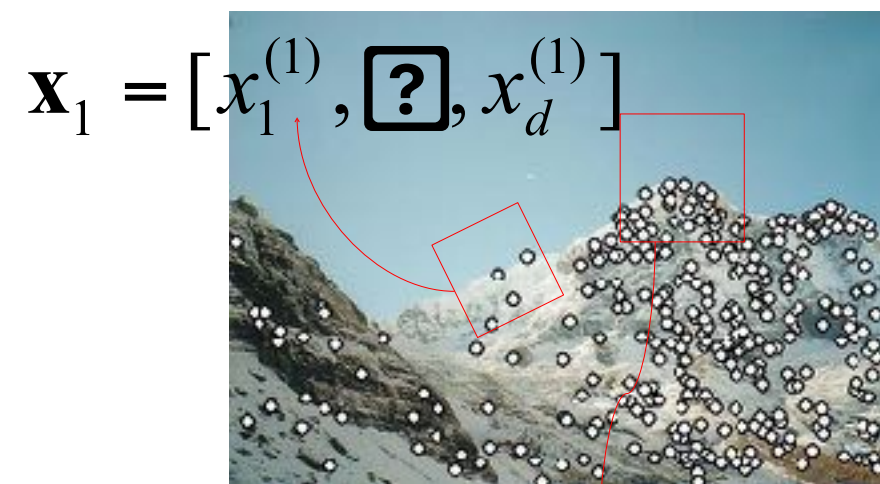
Inverted DoG Kernel (sigma1=5.0, sigma2=6.0)



Scale-invariant feature transform (SIFT, Lowe 2004)

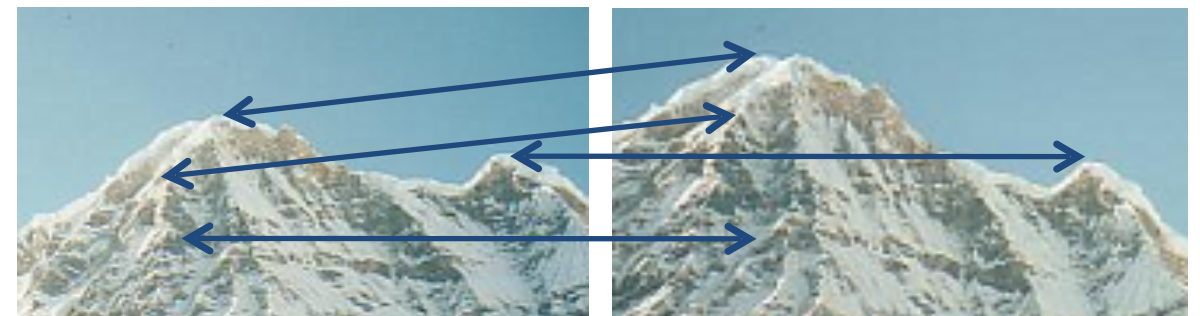
Feature detection and matching

- 1) Detection:** Identify the interest points
- 2) Description:** Extract vector feature descriptor surrounding each interest point.
- 3) Matching:** Determine correspondence between descriptors in two views



$$\mathbf{x}_1 = [x_1^{(1)}, \boxed{?}, x_d^{(1)}]$$

$$\mathbf{x}_2 = [x_1^{(2)}, \boxed{?}, x_d^{(2)}]$$



Object Recognition from Local Scale-Invariant Features

David G. Lowe
Computer Science Department
University of British Columbia
Vancouver, B.C., V6T 1Z4, Canada
lowe@cs.ubc.ca

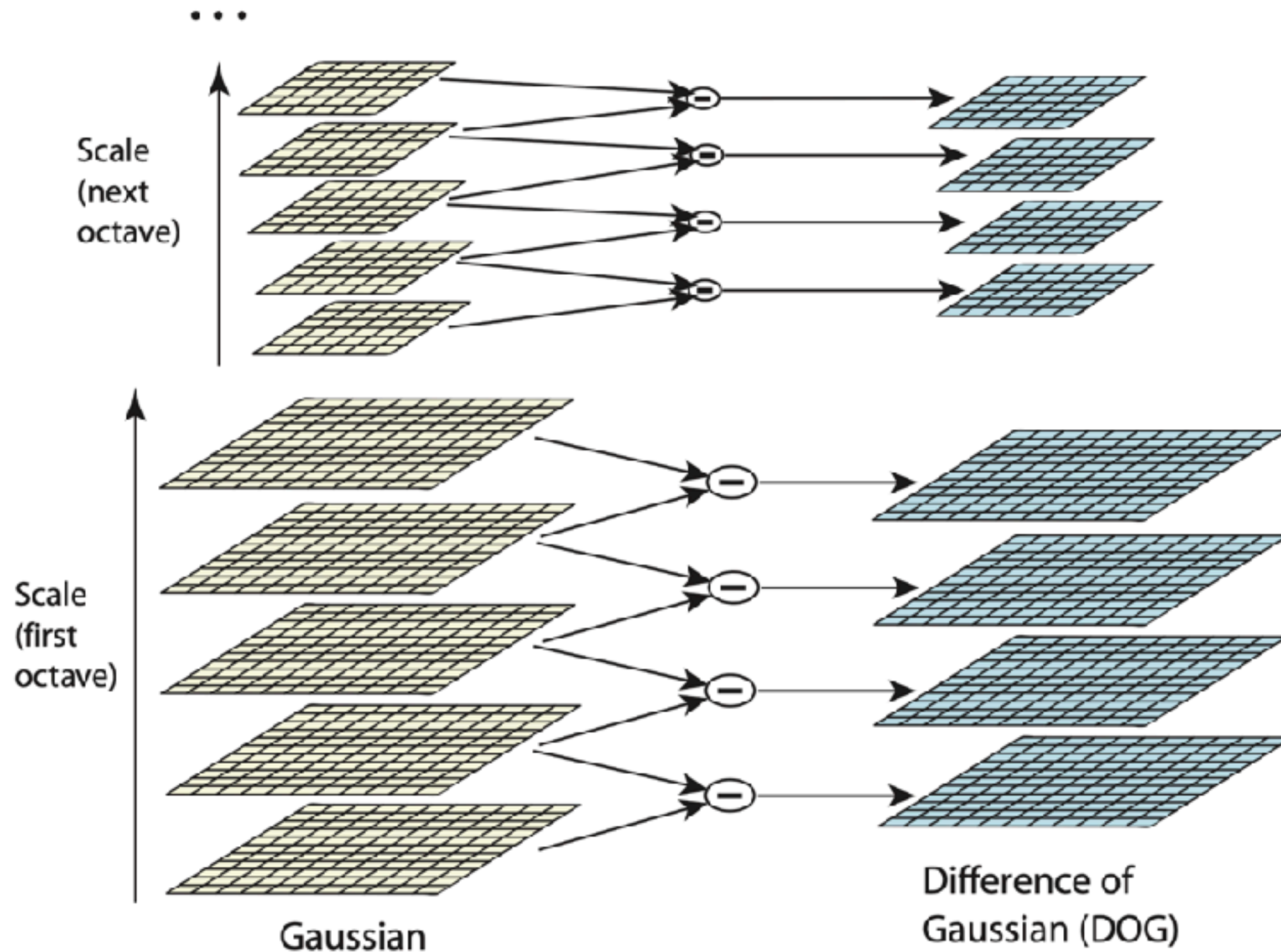
1999

Distinctive Image Features from Scale-Invariant Keypoints

David G. Lowe
Computer Science Department
University of British Columbia
Vancouver, B.C., Canada
lowe@cs.ubc.ca

2004

Difference-of-Gaussian scale space



Local maxima in the 3D DoG pyramid

- Use points where the DoG value is (significantly) lower or higher than all neighbors in the 3x3x3 neighborhood

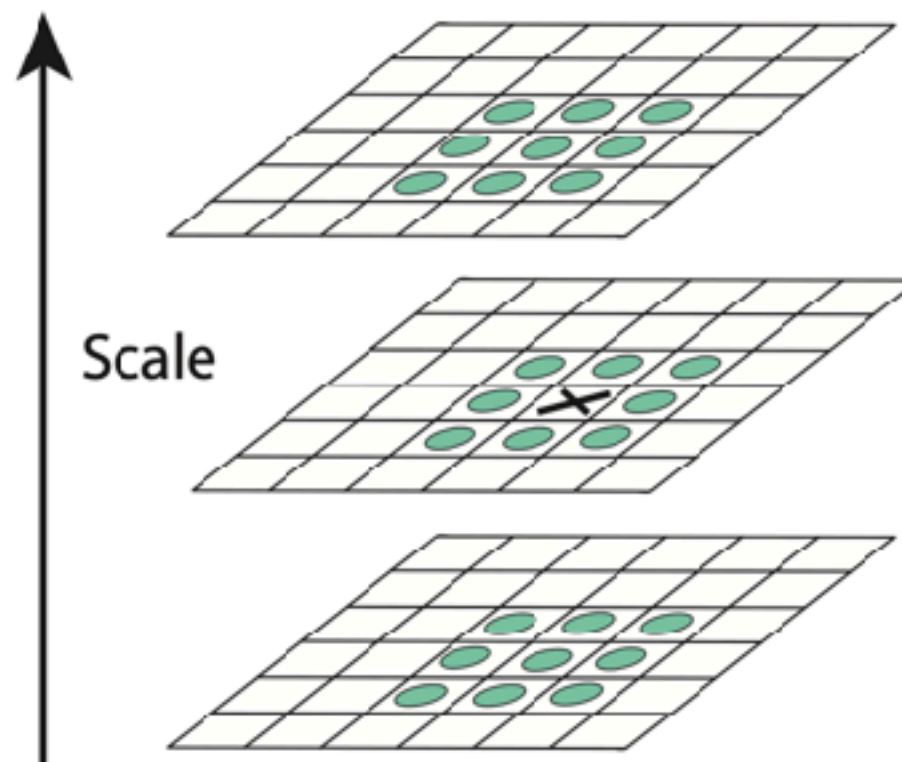
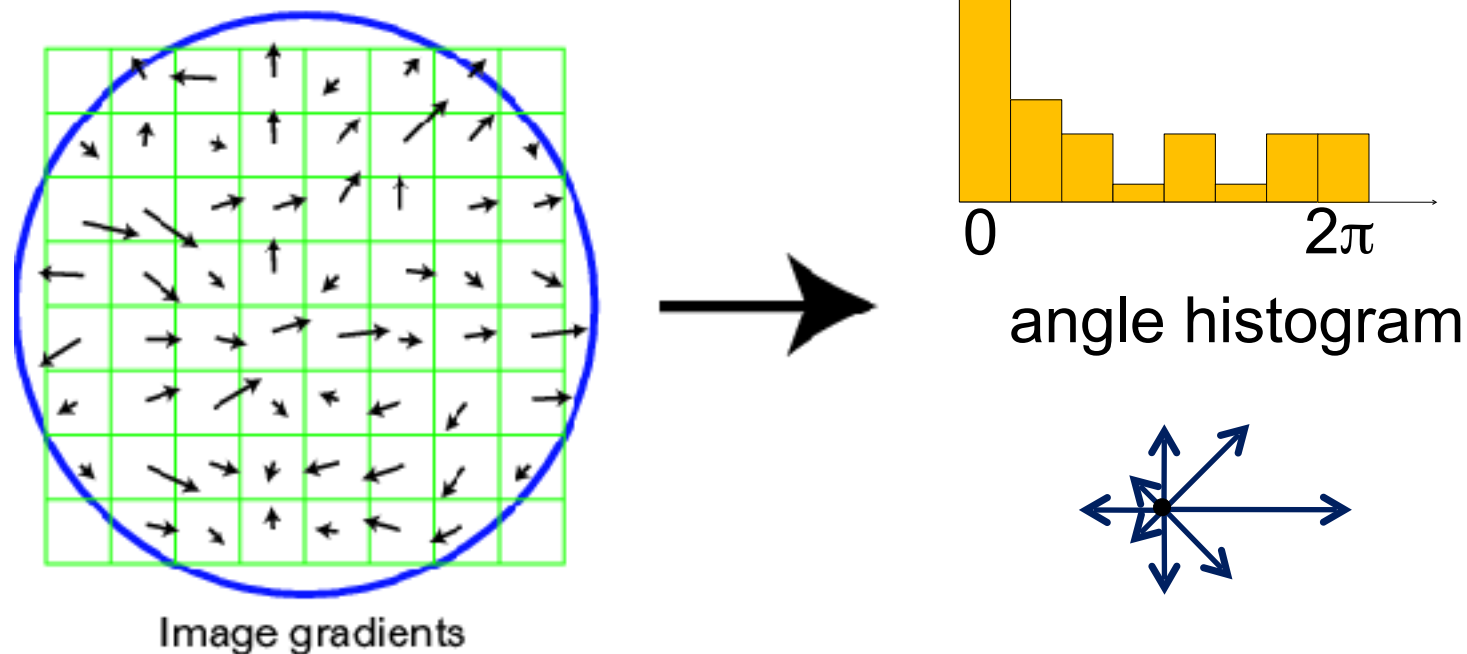


Figure 2: Maxima and minima of the difference-of-Gaussian images are detected by comparing a pixel (marked with X) to its 26 neighbors in 3x3 regions at the current and adjacent scales (marked with circles).

3) Create a descriptor for the window

Basic idea:

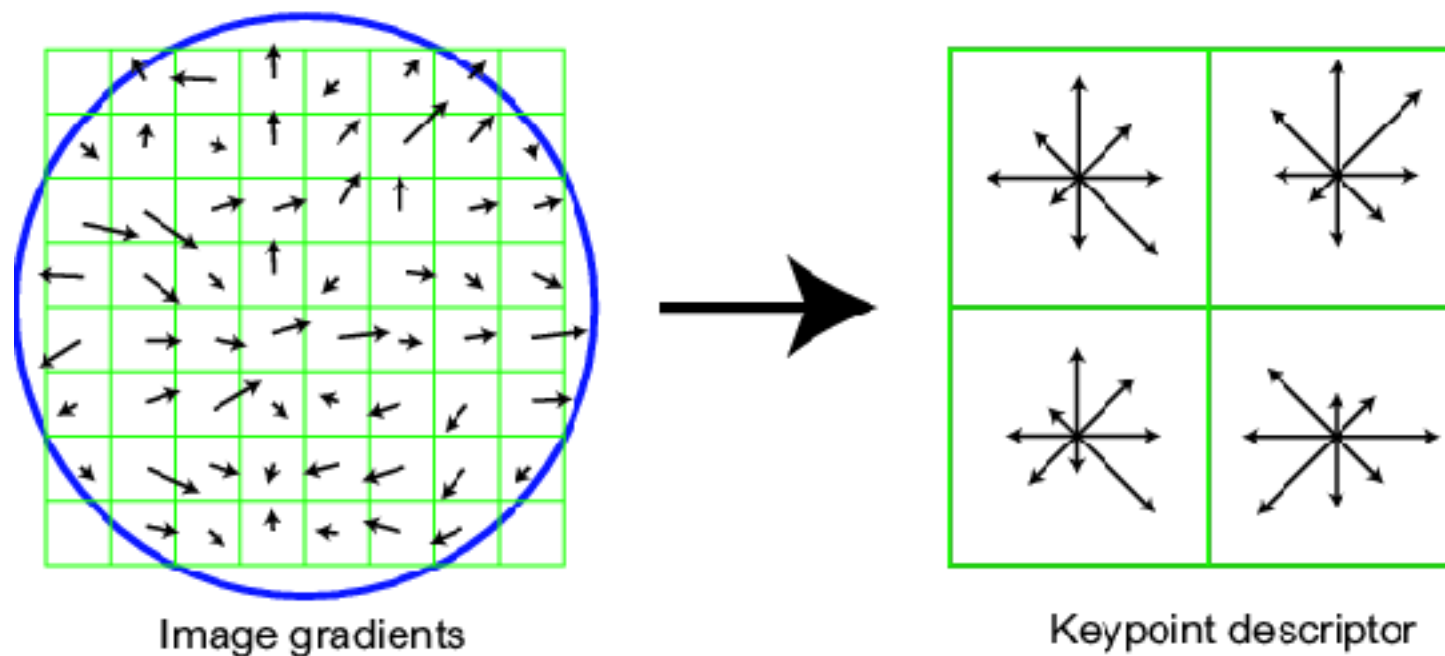
- Take 16x16 square window around detected feature
- Compute edge orientation (angle of the gradient - 90°) for each pixel
- Throw out weak edges (threshold gradient magnitude)
- Create histogram of surviving edge orientations



3) Create a descriptor for the window

Full version

- Divide the 16x16 window into a 4x4 grid of cells (2x2 case shown below)
- Compute an orientation histogram for each cell
- 16 cells * 8 orientations = 128 dimensional descriptor



**A different approach:
Finding well localizable windows
with the structure tensor**

What are good windows?

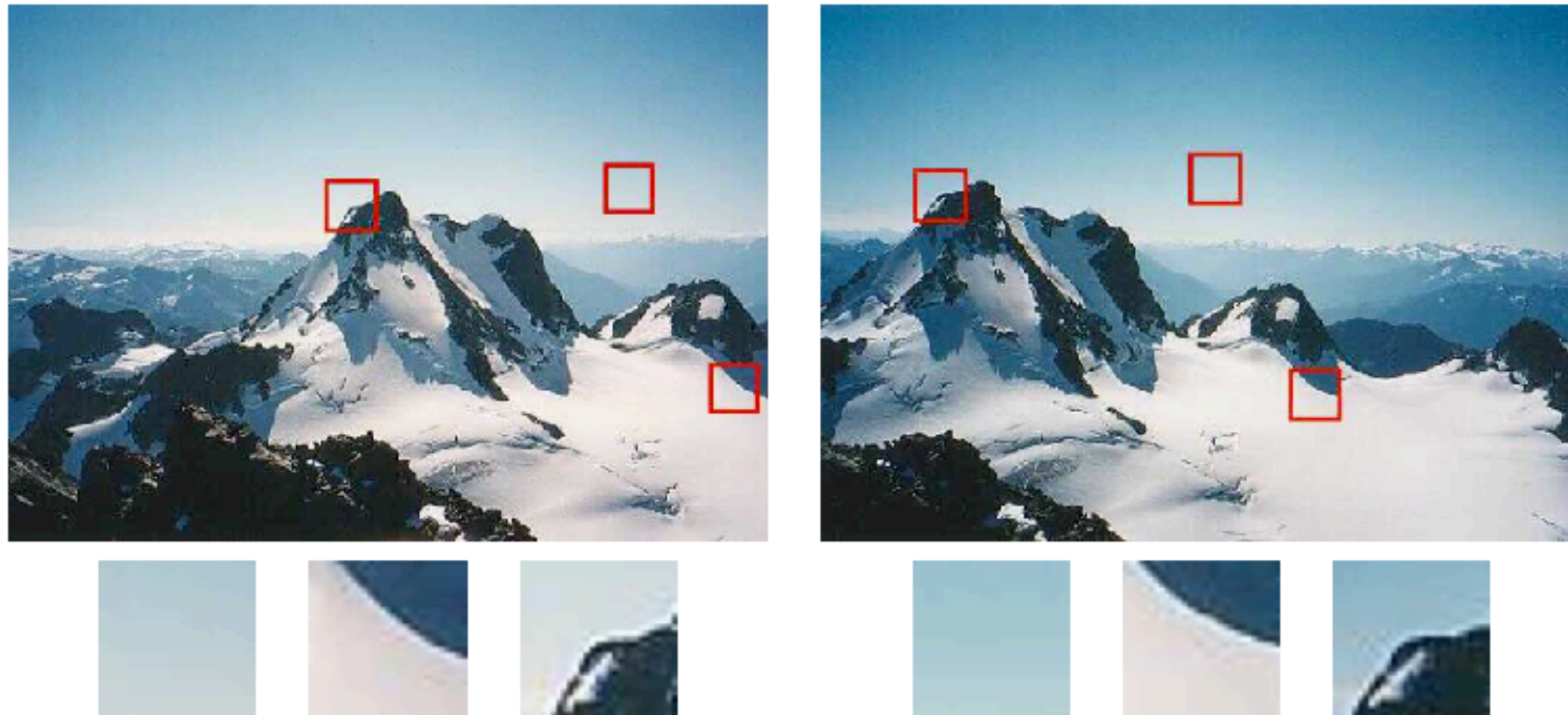


Figure 7.3 *Image pairs with extracted patches below. Notice how some patches can be localized or matched with higher accuracy than others.*

Well localizable windows

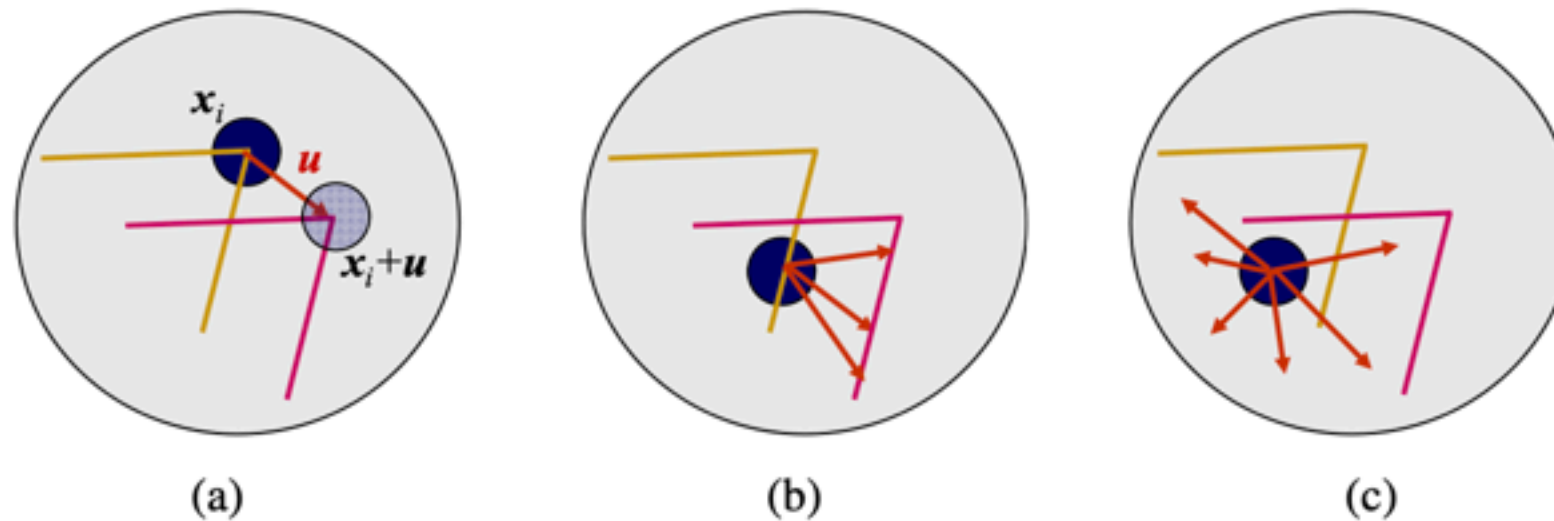


Figure 7.4 Aperture problems for different image patches: (a) stable (“corner-like”) flow; (b) classic aperture problem (barber-pole illusion); (c) textureless region. The two images I_0 (yellow) and I_1 (red) are overlaid. The red vector u indicates the displacement between the patch centers and the $w(x_i)$ weighting function (patch window) is shown as a dark circle.

Well localizable windows

When performing feature detection, we do not know which other image locations the feature will end up being matched against. Therefore, we can only compute how stable this metric is with respect to small variations in position $\Delta \mathbf{u}$ by comparing an image patch against itself, which is known as an *auto-correlation function* or *surface*

$$E_{AC}(\Delta \mathbf{u}) = \sum_i w(\mathbf{x}_i) [I_0(\mathbf{x}_i + \Delta \mathbf{u}) - I_0(\mathbf{x}_i)]^2 \quad (7.2)$$

(Figure 7.5).¹ Note how the auto-correlation surface for the textured flower bed (Figure 7.5b and the red cross in the lower right quadrant of Figure 7.5a) exhibits a strong minimum, indicating that it can be well localized. The correlation surface corresponding to the roof edge (Figure 7.5c) has a strong ambiguity along one direction, while the correlation surface corresponding to the cloud region (Figure 7.5d) has no stable minimum.

Well localizable windows

Using a Taylor Series expansion of the image function $I_0(\mathbf{x}_i + \Delta \mathbf{u}) \approx I_0(\mathbf{x}_i) + \nabla I_0(\mathbf{x}_i) \cdot \Delta \mathbf{u}$ (Lucas and Kanade 1981; Shi and Tomasi 1994), we can approximate the auto-correlation surface as

$$E_{AC}(\Delta \mathbf{u}) = \sum_i w(\mathbf{x}_i) [I_0(\mathbf{x}_i + \Delta \mathbf{u}) - I_0(\mathbf{x}_i)]^2 \quad (7.3)$$

$$\approx \sum_i w(\mathbf{x}_i) [I_0(\mathbf{x}_i) + \nabla I_0(\mathbf{x}_i) \cdot \Delta \mathbf{u} - I_0(\mathbf{x}_i)]^2 \quad (7.4)$$

$$= \sum_i w(\mathbf{x}_i) [\nabla I_0(\mathbf{x}_i) \cdot \Delta \mathbf{u}]^2 \quad (7.5)$$

$$= \Delta \mathbf{u}^T \mathbf{A} \Delta \mathbf{u}, \quad (7.6)$$

where

$$\nabla I_0(\mathbf{x}_i) = \left(\frac{\partial I_0}{\partial x}, \frac{\partial I_0}{\partial y} \right)(\mathbf{x}_i) \quad (7.7)$$

is the *image gradient* at $\mathbf{x}_i$. This gradient can be computed using a variety of techniques (Schmid, Mohr, and Bauckhage 2000). The classic “Harris” detector (Harris and Stephens 1988) uses a $[-2 \ -1 \ 0 \ 1 \ 2]$ filter, but more modern variants (Schmid, Mohr, and Bauckhage 2000; Triggs 2004) convolve the image with horizontal and vertical derivatives of a Gaussian (typically with $\sigma = 1$).

Well localizable windows

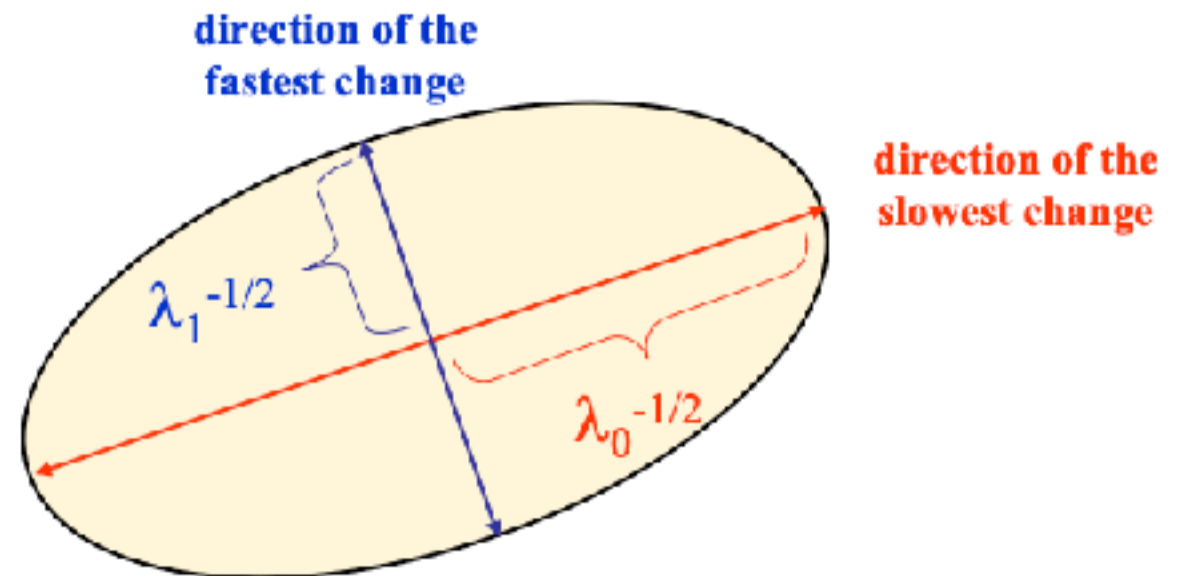
The auto-correlation matrix $\mathbf{A}$ can be written as

$$\mathbf{A} = w * \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}, \quad (7.8)$$

where we have replaced the weighted summations with discrete convolutions with the weighting kernel w . This matrix can be interpreted as a tensor (multiband) image, where the outer products of the gradients ∇I are convolved with a weighting function w to provide a per-pixel estimate of the local (quadratic) shape of the auto-correlation function.

The autocorrelation matrix

$$\mathbf{A} = w * \begin{bmatrix} f_x^2 & f_{xy} \\ f_{xy} & f_y^2 \end{bmatrix}$$



- Inverse provides lower bound on the uncertainty of the location of the window
- Eigenvalues λ_0, λ_1 of $\mathbf{A}$ are interesting - smaller eigenvalue indicates largest uncertainty

Detecting interpretable and accurate scale-invariant keypoints

Publisher: IEEE

Cite This

PDF

Wolfgang Förstner ; Timo Dickscheid ; Falko Schindler All Authors

2. Related work

The *basic local feature detectors* by Förstner [5] and Harris and Stephens [7] detect windows of given size having locally maximal localisation accuracy with respect to least-squares matching onto a shifted, noisy copy of themselves. This is achieved by evaluating the structure tensor, or second moment matrix

$$M_{\sigma,\tau} = \overline{\nabla_{\tau} g \nabla_{\tau} g^T} = G_{\sigma} * \begin{bmatrix} g_{x,\tau}^2 & g_{x,\tau} g_{y,\tau} \\ g_{x,\tau} g_{y,\tau} & g_{y,\tau}^2 \end{bmatrix} \quad (1)$$

which is computed by first determining the gradient $\nabla_{\tau} g = [g_{x,\tau}, g_{y,\tau}]^T$ using the *differentiation scale* τ , and then taking the average with a kernel having *integration scale* σ , which is usually chosen as a Gaussian. [19] have shown that selecting windows is most effective when using local maxima of the smallest eigenvalue $\lambda_2(M)$ above a threshold, thus requiring a sufficiently large gradient content within the window to be locally optimal. This is both in contrast to the proposals in [5], where a minimum ratio λ_2/λ_1 is required for excluding points on edges, and the heuristic measure $\det M_{\sigma,\tau} - k \operatorname{tr}^2 M_{\sigma,\tau}$ in [7], where k is determined empirically. As shown in [5], the detected windows are not only highly suitable for matching, but also optimal for locating centres of junctions and circular symmetric features.

[5]

ISPRS Intercommission Workshop, Interlaken, June 1987

A Fast Operator for Detection and Precise Location of Distinct Points, Corners and Centres of Circular Features

by W. Förstner and E. Gülch
Institute for Photogrammetry
Stuttgart University

[19]

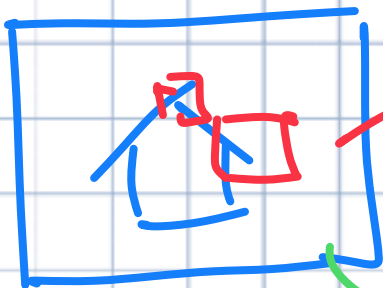
Good Features to Track

Jianbo Shi and Carlo Tomasi¹

December 1993

Feature descriptor matching

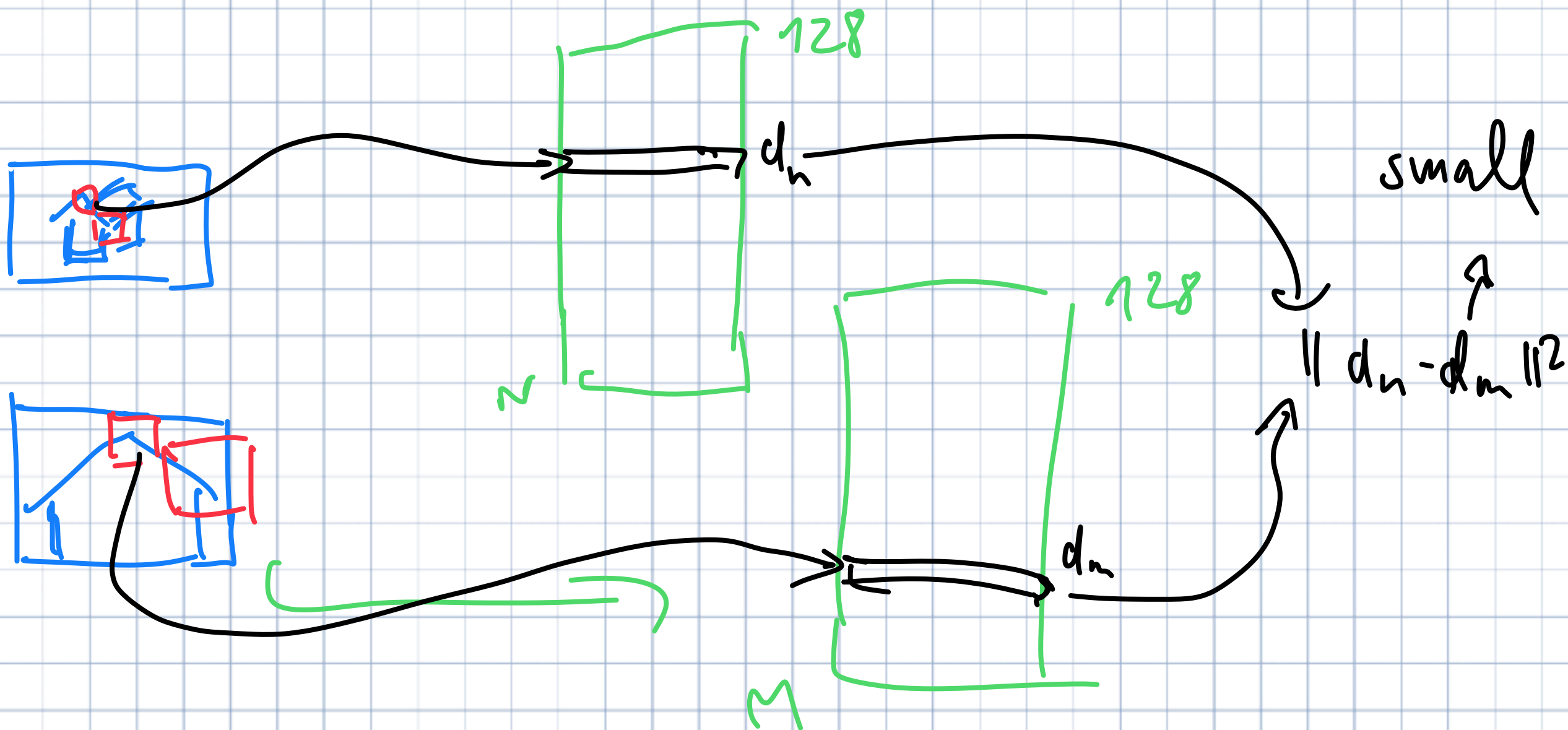
Image



1) Detection, e.g. Laplacian operator

Windows: $[(x_1, y_1, s_1), (x_2, y_2, s_2) \dots]$

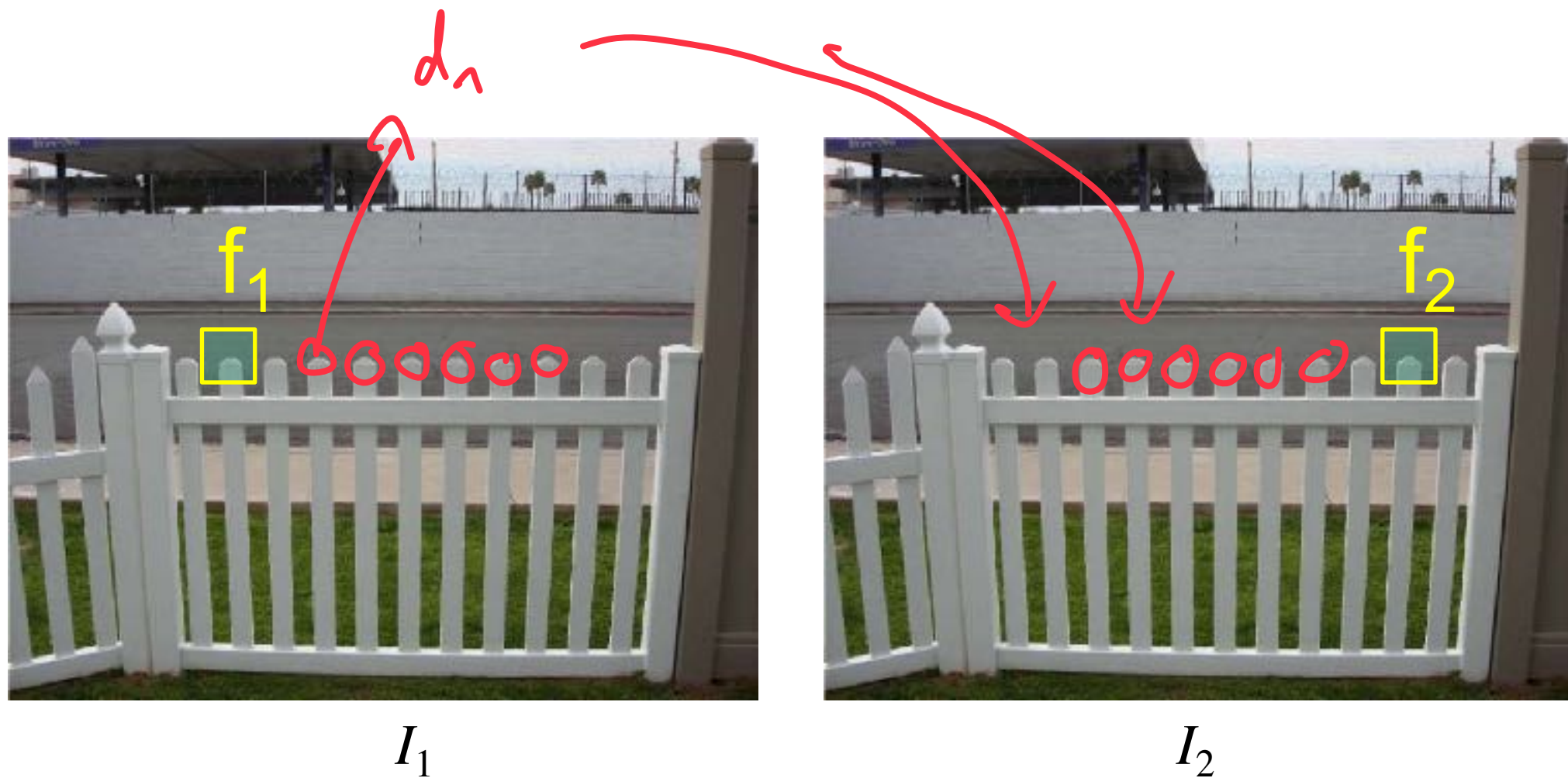
2) Description



Feature distance

How to define the difference between two features f_1, f_2 ?

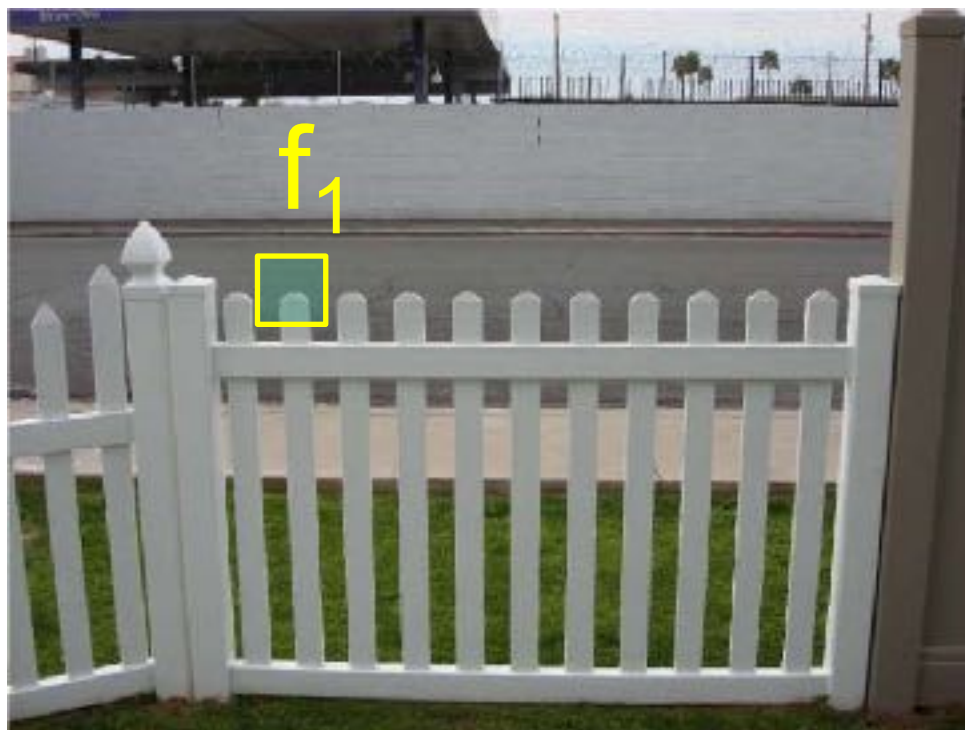
- Simple approach: L_2 distance, $\| f_1 - f_2 \|$



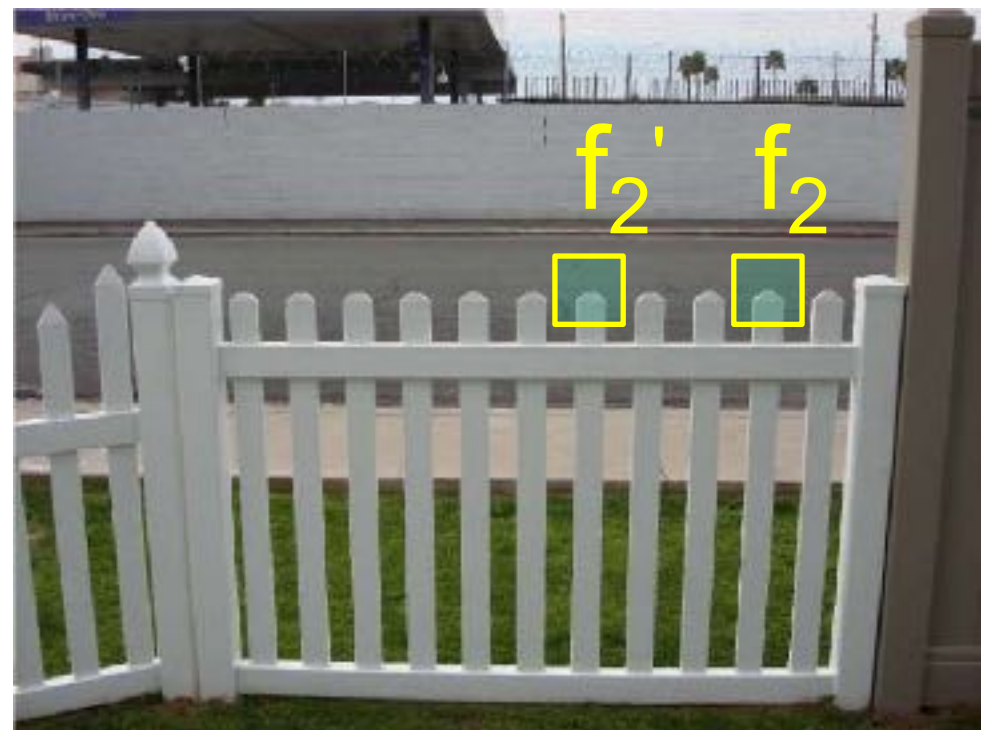
Feature distance

How to define the difference between two features f_1, f_2 ?

- Better approach: ratio distance = $\|f_1 - f_2\| / \|f_1 - f_2'\|$
 - f_2 is the best SSD match to f_1 in I_2
 - f_2' is the 2nd best SSD match to f_1 in I_2
 - gives large values for ambiguous matches



I_1



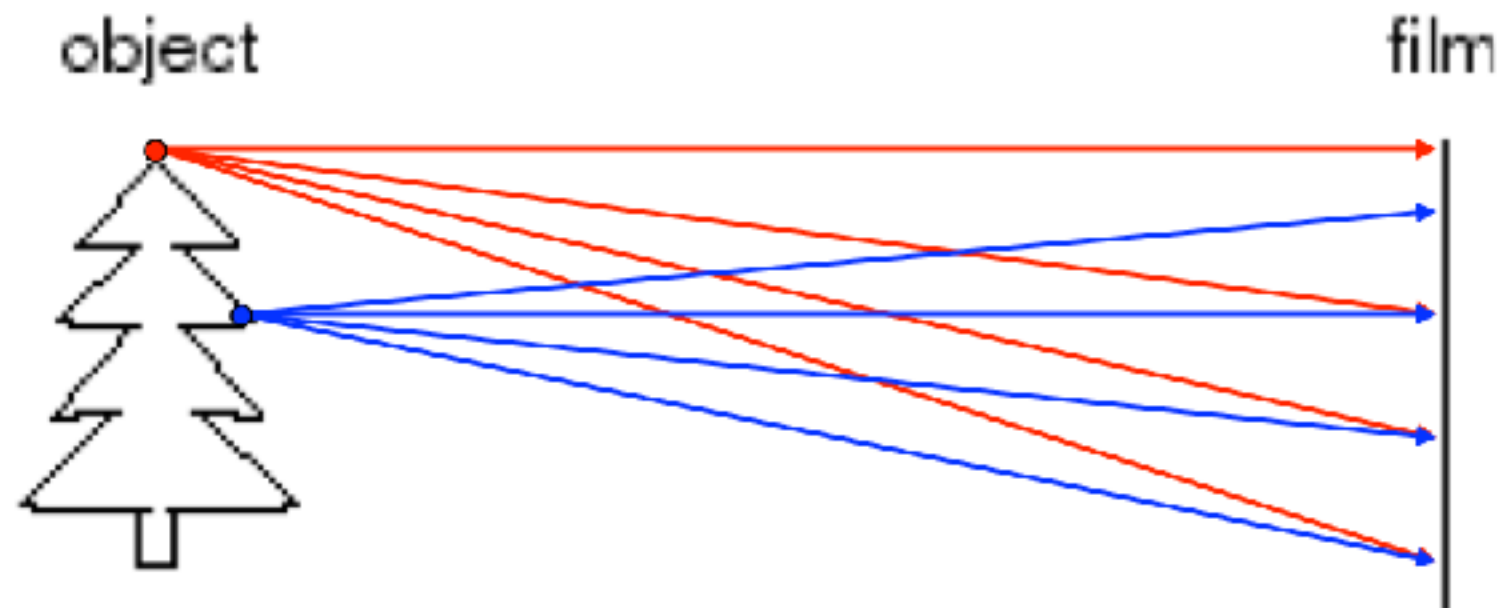
I_2

Back to cameras

Let's design a camera

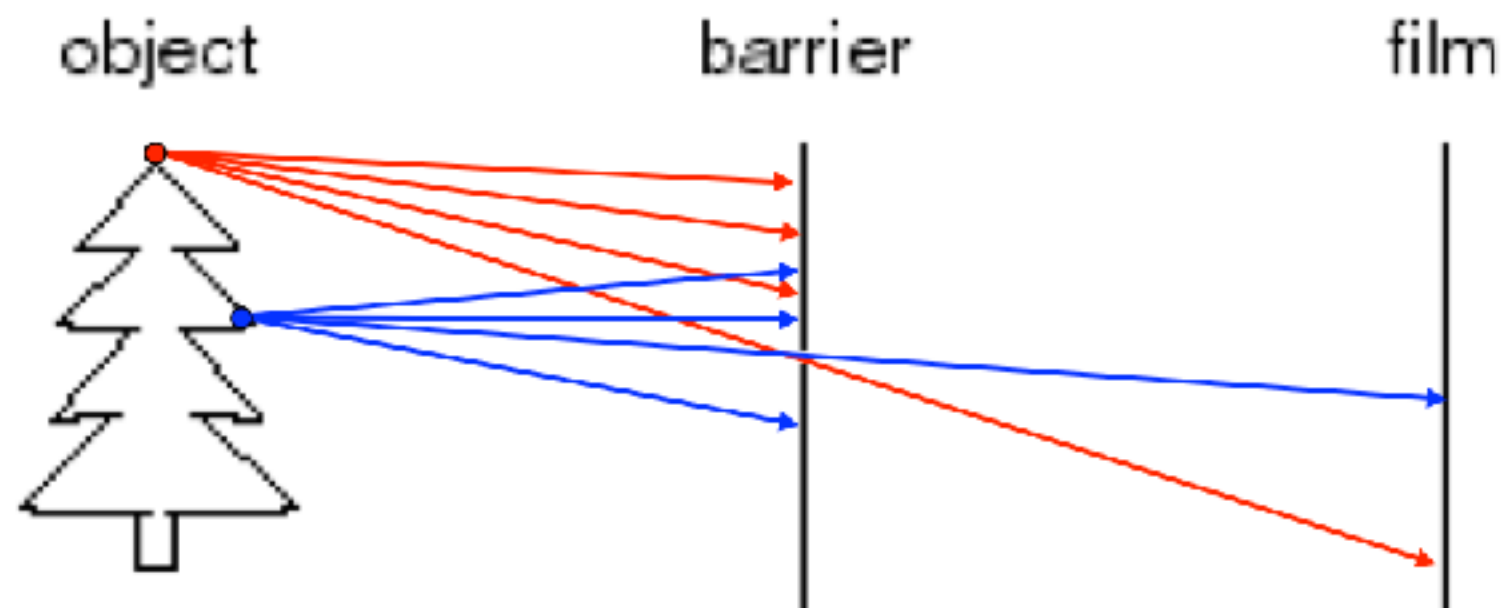
Idea 1: put a piece of film in front of an object
Do we get a reasonable image?

Let's design a camera



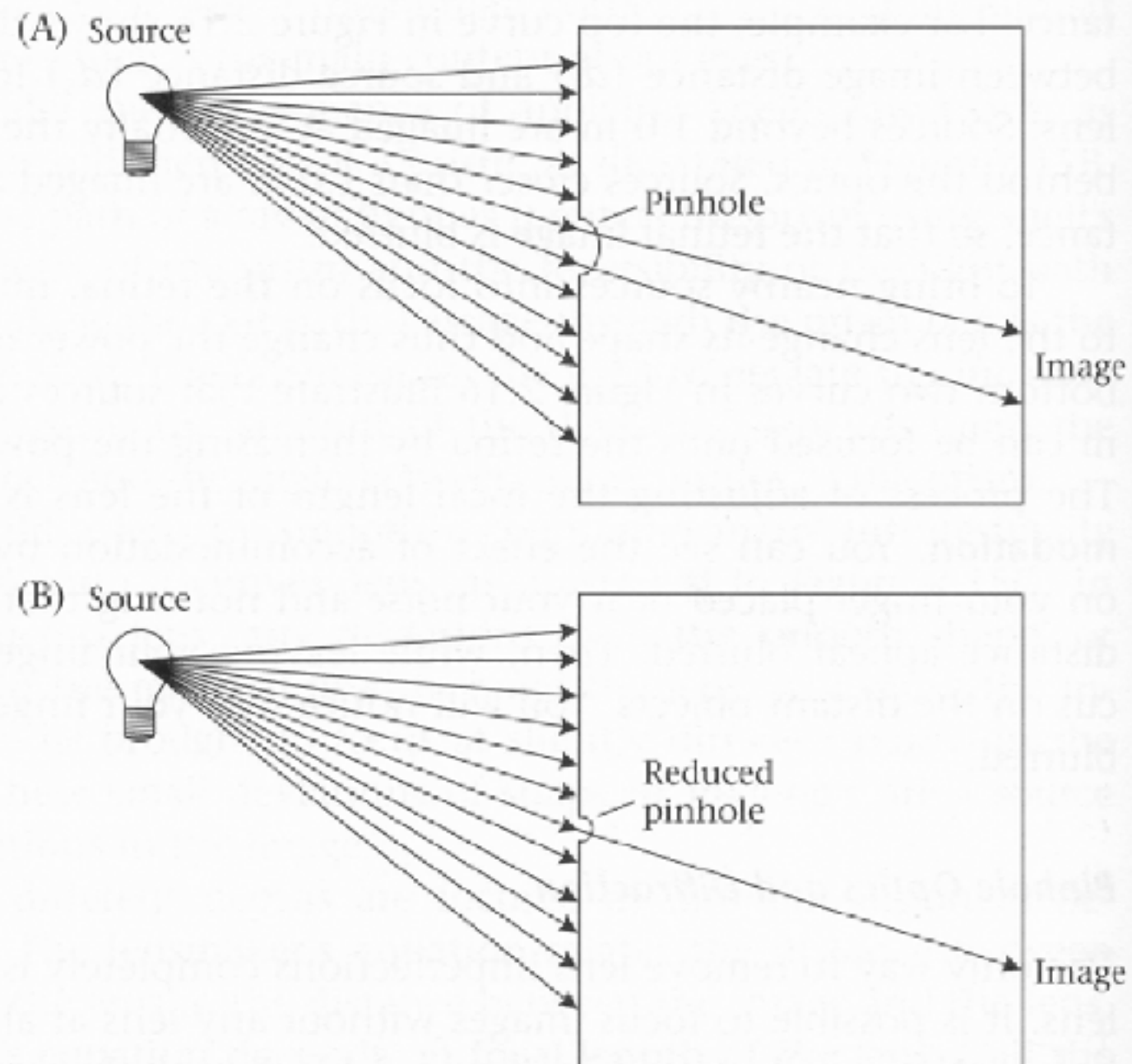
Idea 1: put a piece of film in front of an object
Do we get a reasonable image?

Pinhole camera

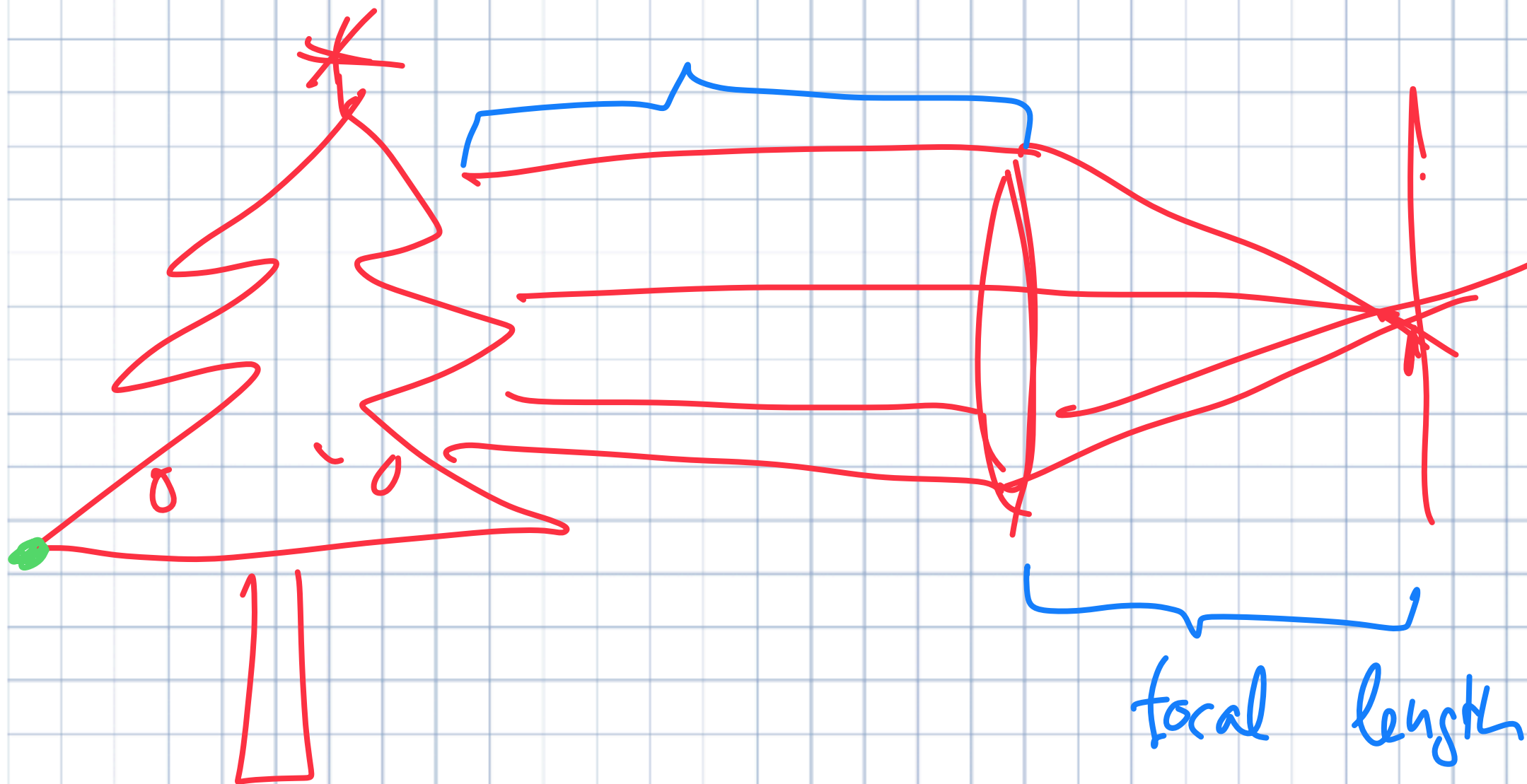


Add a barrier to block off most of the rays

Effect of pinhole size

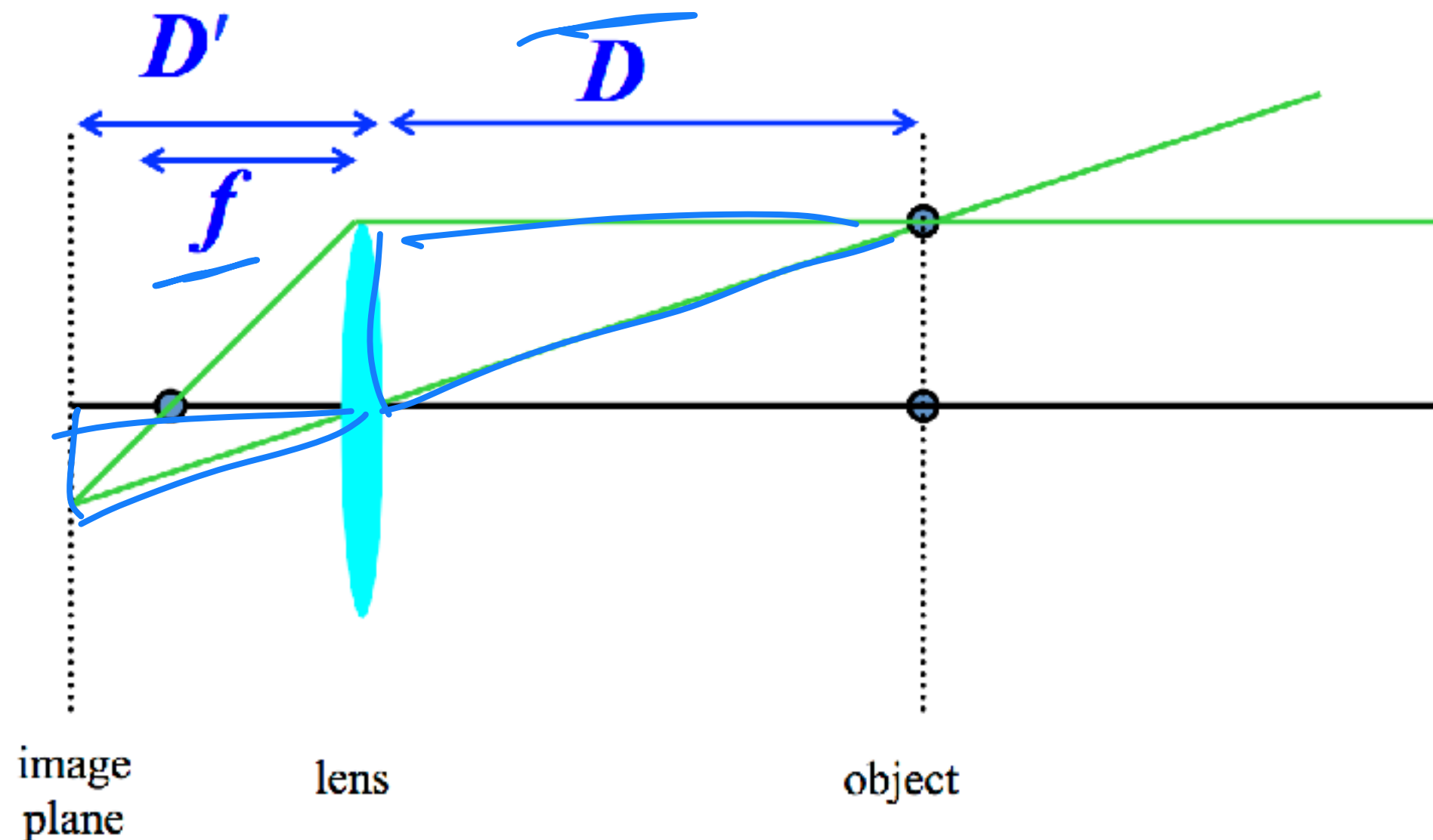


Wandell, Foundations of Vision, Sinauer, 1995



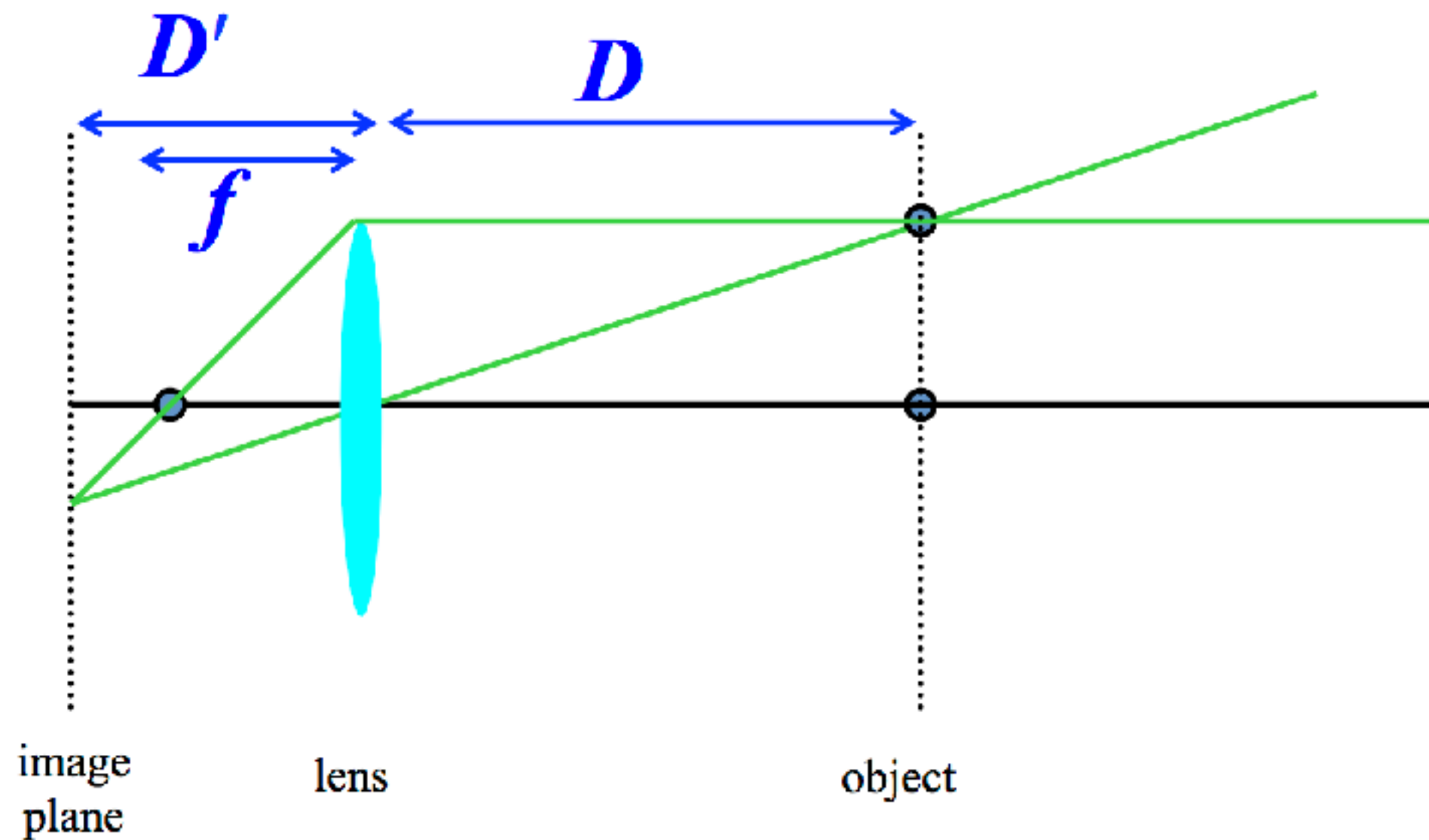
Thin lens formula

- What is the relation between the focal length (f), the distance of the object from the optical center (D), and the distance at which the object will be in focus (D')?



Thin lens formula

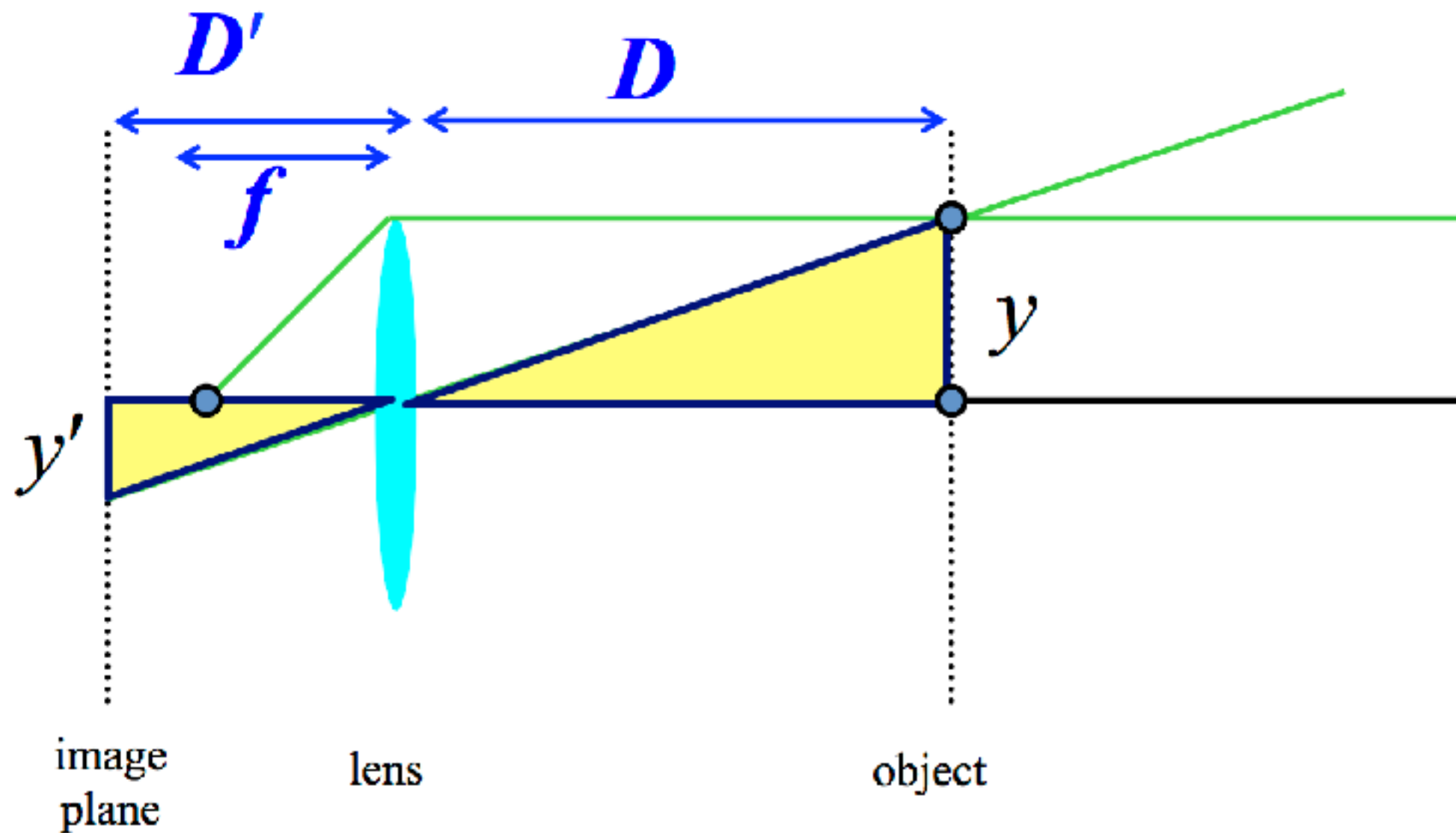
Similar triangles everywhere!



Thin lens formula

Similar triangles everywhere!

$$y'/y = D'/D$$

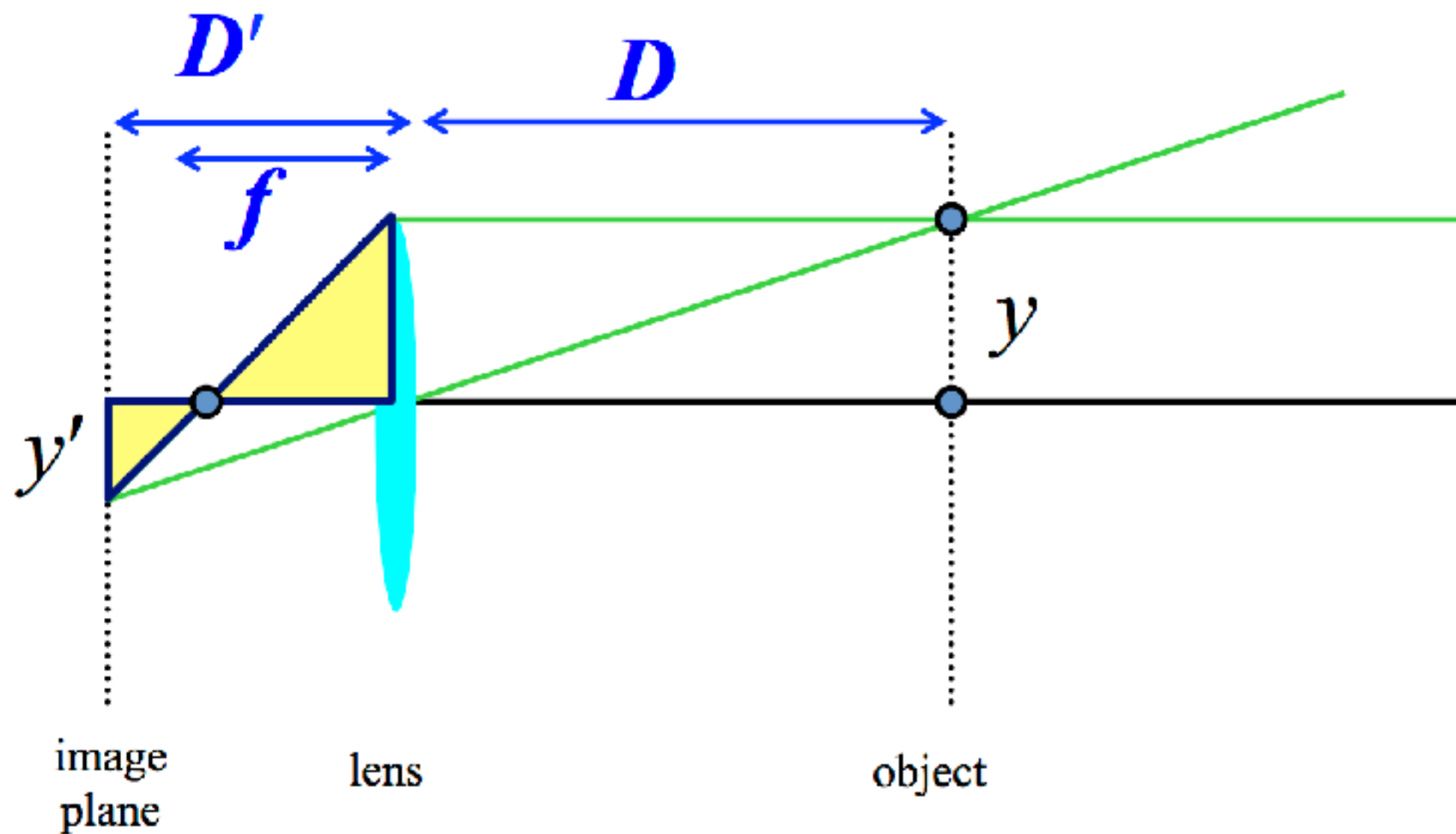


Thin lens formula

Similar triangles everywhere!

$$y'/y = D'/D$$

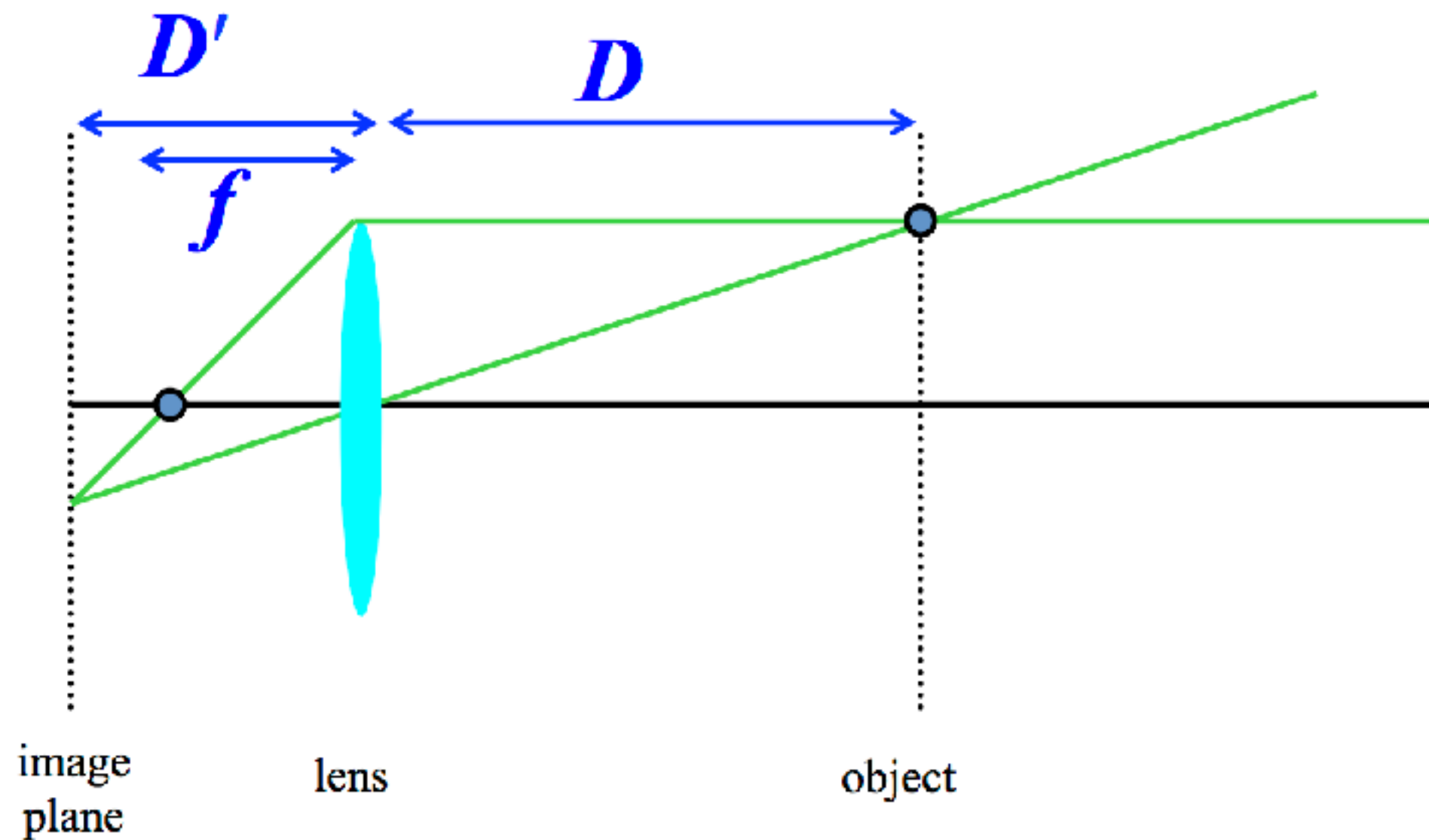
$$y'/y = (D' - f)/f$$



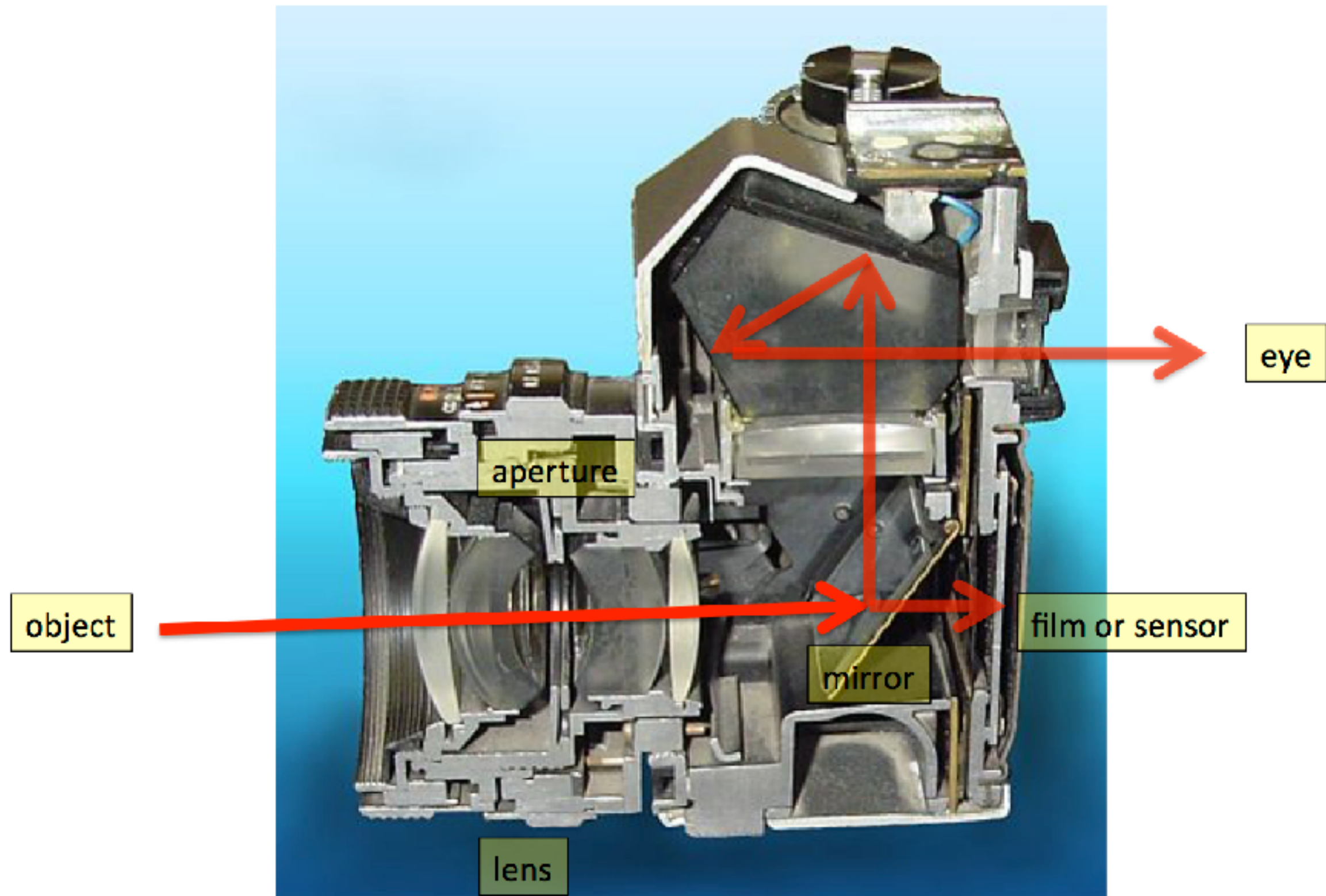
Thin lens formula

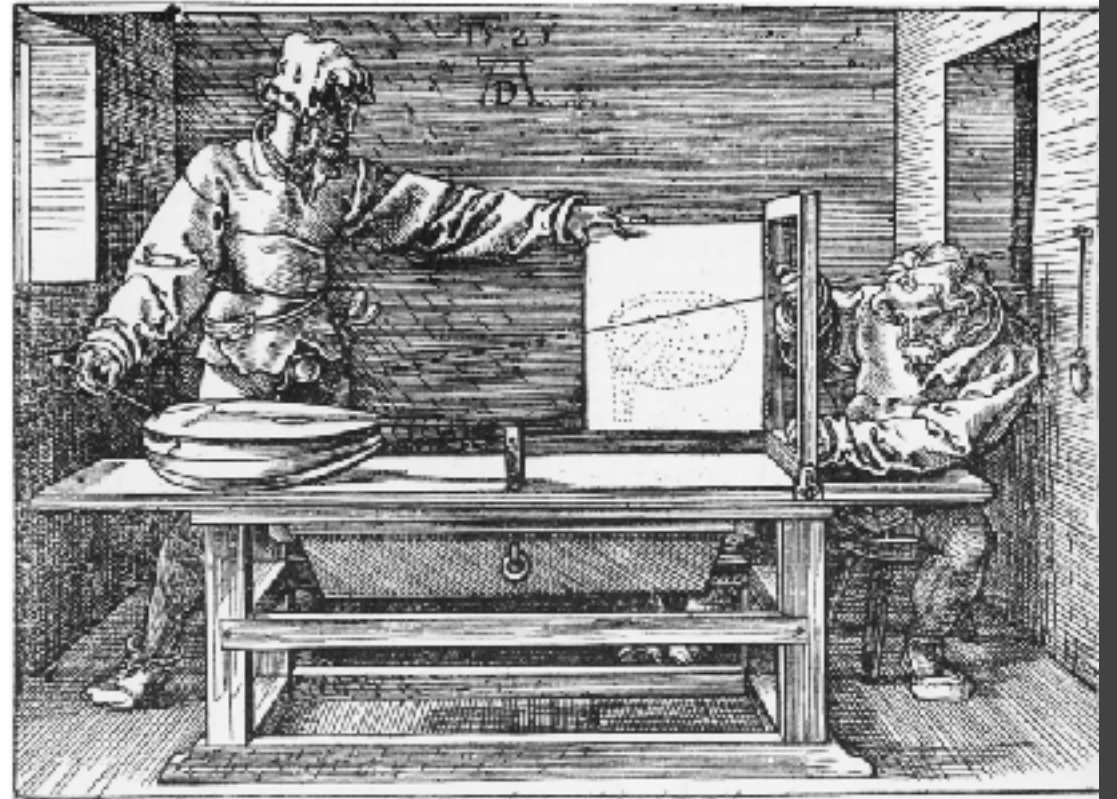
$$\frac{1}{D'} + \frac{1}{D} = \frac{1}{f}$$

Any point satisfying the thin lens equation is in focus.



Single-lens reflex camera (SLR)



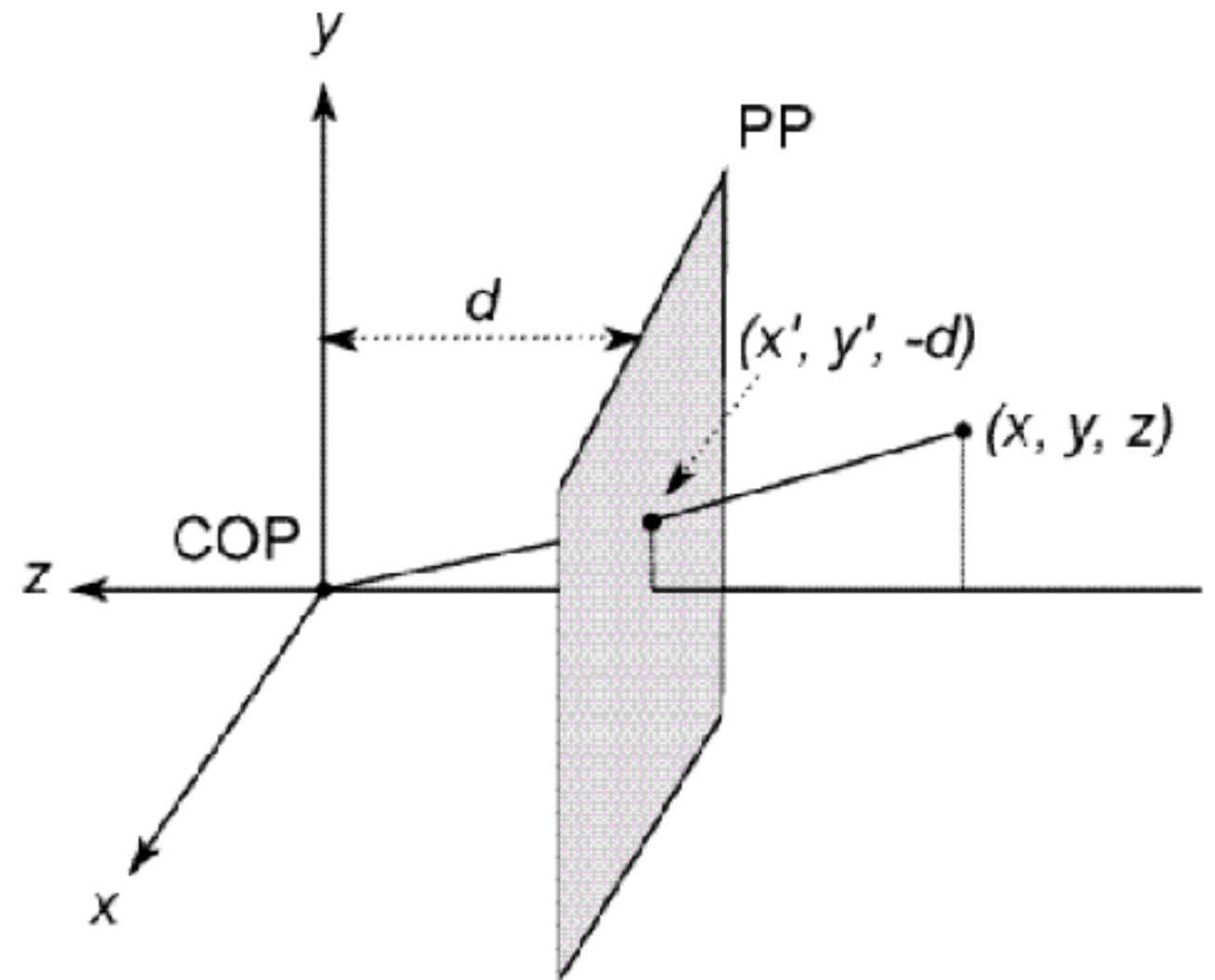


Albrecht Dürer, *Mechanical creation of a perspective image*, 1525

Perspective projection

Projecting a point

- The origin is the center of projection (COP)
- We model the image plane (projection plane) at $z = -d$ in front of the camera



Projecting a point

- We observe a point $P = (x, y, z)$ projecting to $p = (x', y', -d)$

What are x' and y' ?

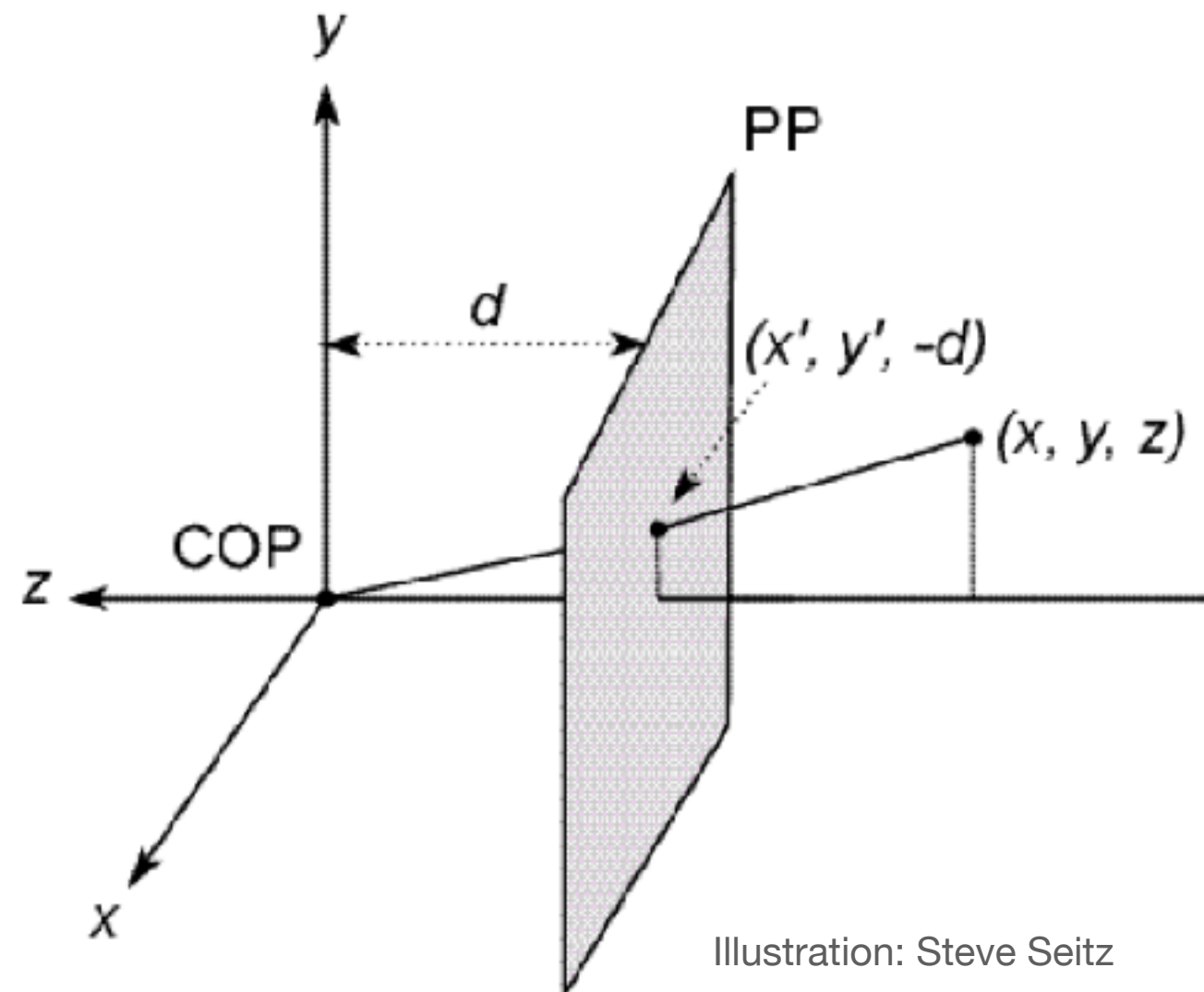


Illustration: Steve Seitz

Projecting a point

- We observe a point $P = (x, y, z)$ projecting to $p = (x', y', -d)$
- We have

$$\frac{z}{-d} = \frac{x}{x'} \quad \Leftrightarrow \quad x' = -d \cdot \frac{x}{z}$$

- Accordingly,

$$y' = -d \cdot \frac{y}{z}$$

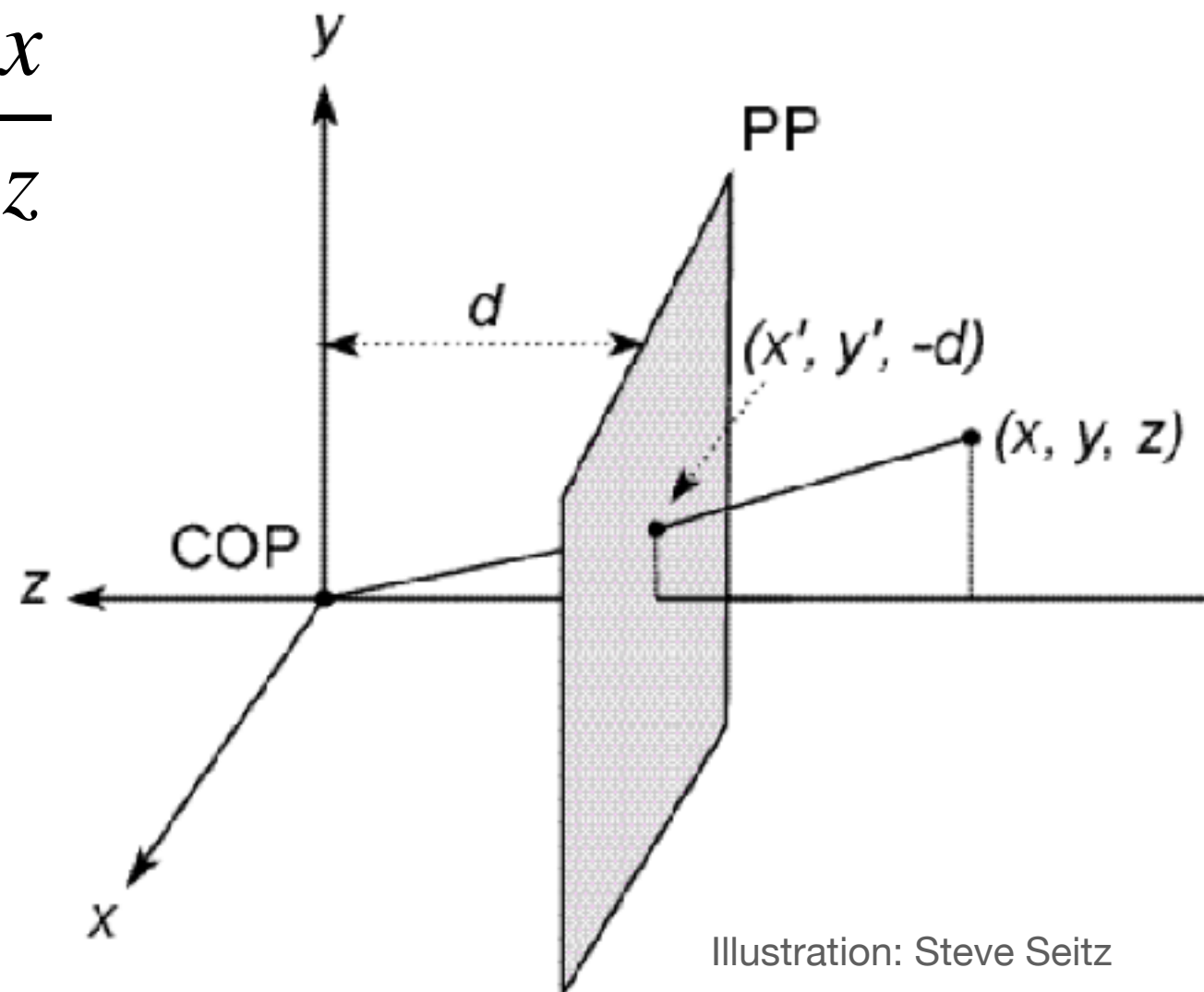


Illustration: Steve Seitz

Projecting a point

Thus we map $(x, y, z) \rightarrow (x \frac{-d}{z}, y \frac{-d}{z})$.

Using homogeneous coordinates, we can write

$$\begin{bmatrix} x \\ y \\ -z/d \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1/d & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

This is equivalent to the normalized coordinate in the Cartesian image plane:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x \cdot \frac{-d}{z} \\ y \cdot \frac{-d}{z} \\ 1 \end{bmatrix} \approx \begin{bmatrix} x \\ y \\ -\frac{z}{d} \end{bmatrix}$$

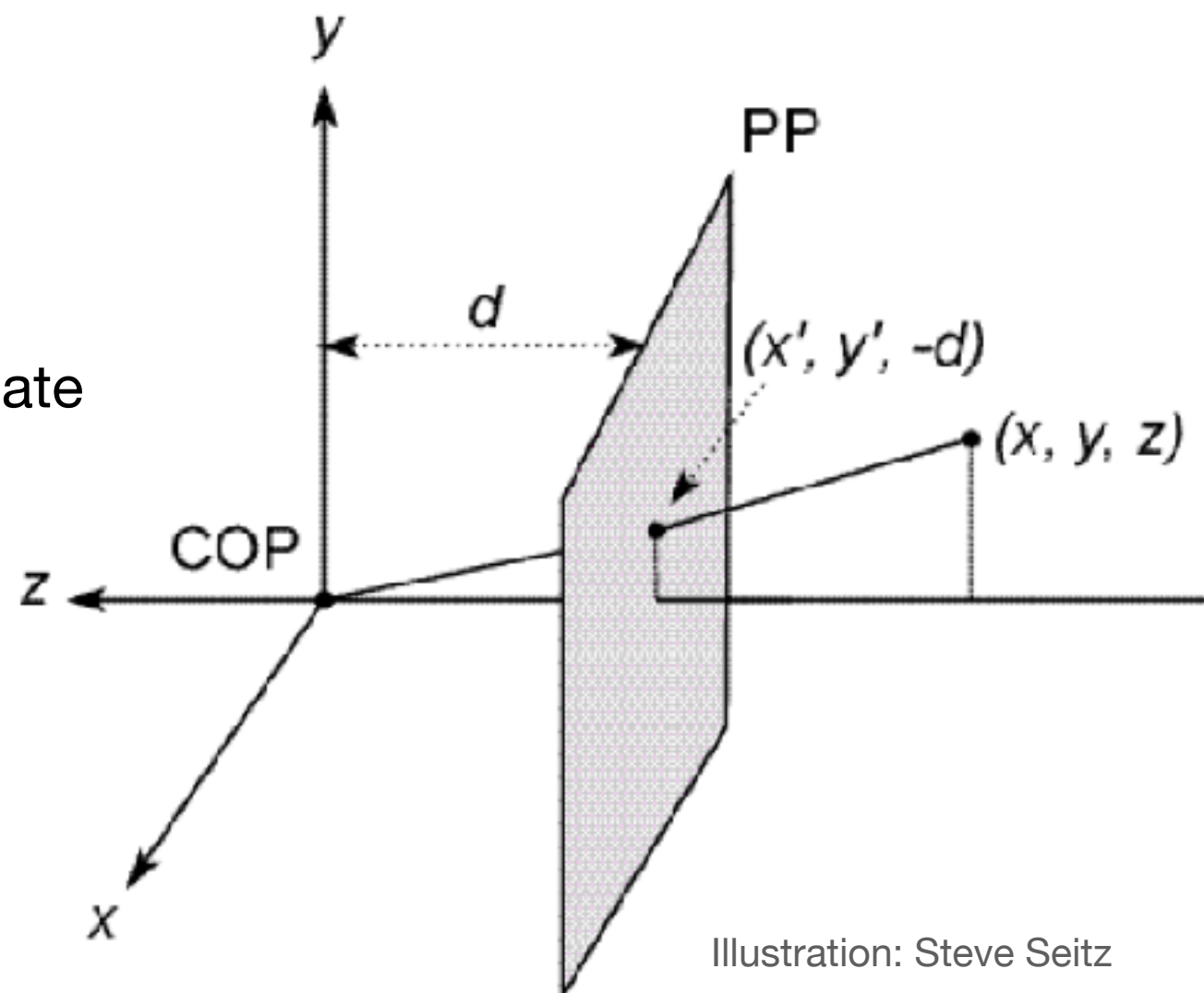


Illustration: Steve Seitz

Many coordinate systems!

- World
- Camera
- Image
- Pixel

Perspective Projection Matrix

Projection is a matrix multiplication using homogeneous coordinates

$$\begin{bmatrix} \\ \\ \\ \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} \\ \\ \\ \end{bmatrix} \Rightarrow \left(f \frac{x}{z}, f \frac{y}{z} \right)$$

divide by the third coordinate

In practice: lots of coordinate transformations...

$$\begin{pmatrix} \text{2D point} \\ (3 \times 1) \end{pmatrix} = \begin{pmatrix} \text{Camera to pixel coord. trans. matrix} \\ (3 \times 3) \end{pmatrix} \begin{pmatrix} \text{Perspective projection matrix} \\ (3 \times 4) \end{pmatrix} \begin{pmatrix} \text{World to camera coord. trans. matrix} \\ (4 \times 4) \end{pmatrix} \begin{pmatrix} \text{3D point} \\ (4 \times 1) \end{pmatrix}$$

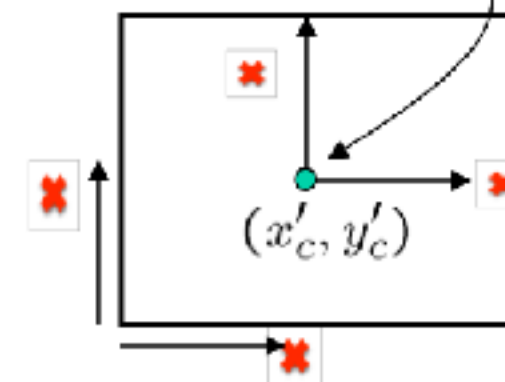
Camera parameters

A camera is described by several parameters

- Translation $\mathbf{T}$ of the optical center from the origin of world coords
- Rotation $\mathbf{R}$ of the image plane
- focal length f , principle point (x'_c, y'_c) , pixel size (s_x, s_y)
- blue parameters are called “**extrinsics**,” red are “**intrinsics**”

Projection equation

$$\mathbf{X} = \begin{bmatrix} sx \\ sy \\ s \end{bmatrix} = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{\Pi} \mathbf{X}$$



- The projection matrix models the cumulative effect of all parameters
- Useful to decompose into a series of operations

$$\mathbf{\Pi} = \underbrace{\begin{bmatrix} -fs_x & 0 & x'_c \\ 0 & -fs_y & y'_c \\ 0 & 0 & 1 \end{bmatrix}}_{\text{intrinsics}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{\text{projection}} \underbrace{\begin{bmatrix} \mathbf{R}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}}_{\text{rotation}} \underbrace{\begin{bmatrix} \mathbf{I}_{3 \times 3} & \mathbf{T}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}}_{\text{translation}}$$

identity matrix

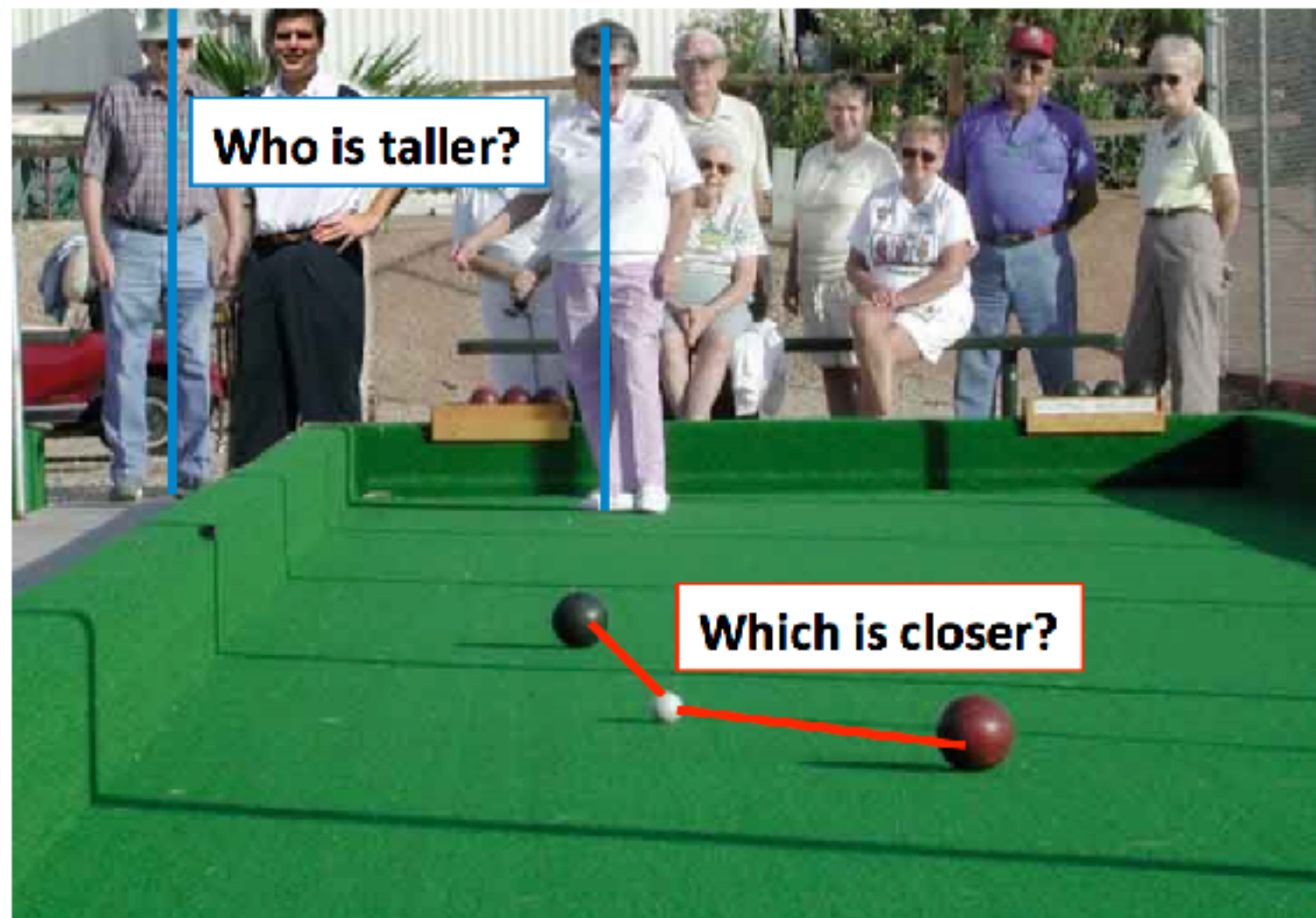
- The definitions of these parameters are **not** completely standardized
 - especially intrinsics—varies from one book to another

Which information gets
lost during projection?

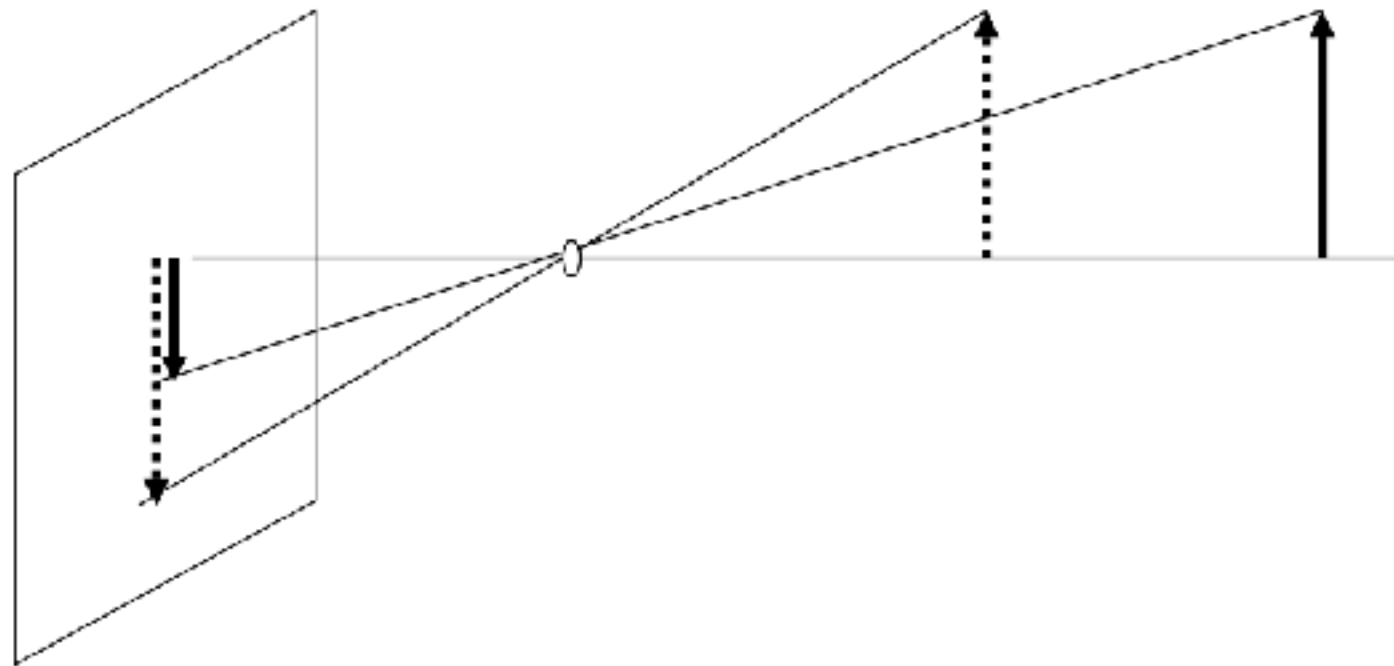
Projective Geometry

What is lost?

- Length



Measuring distance

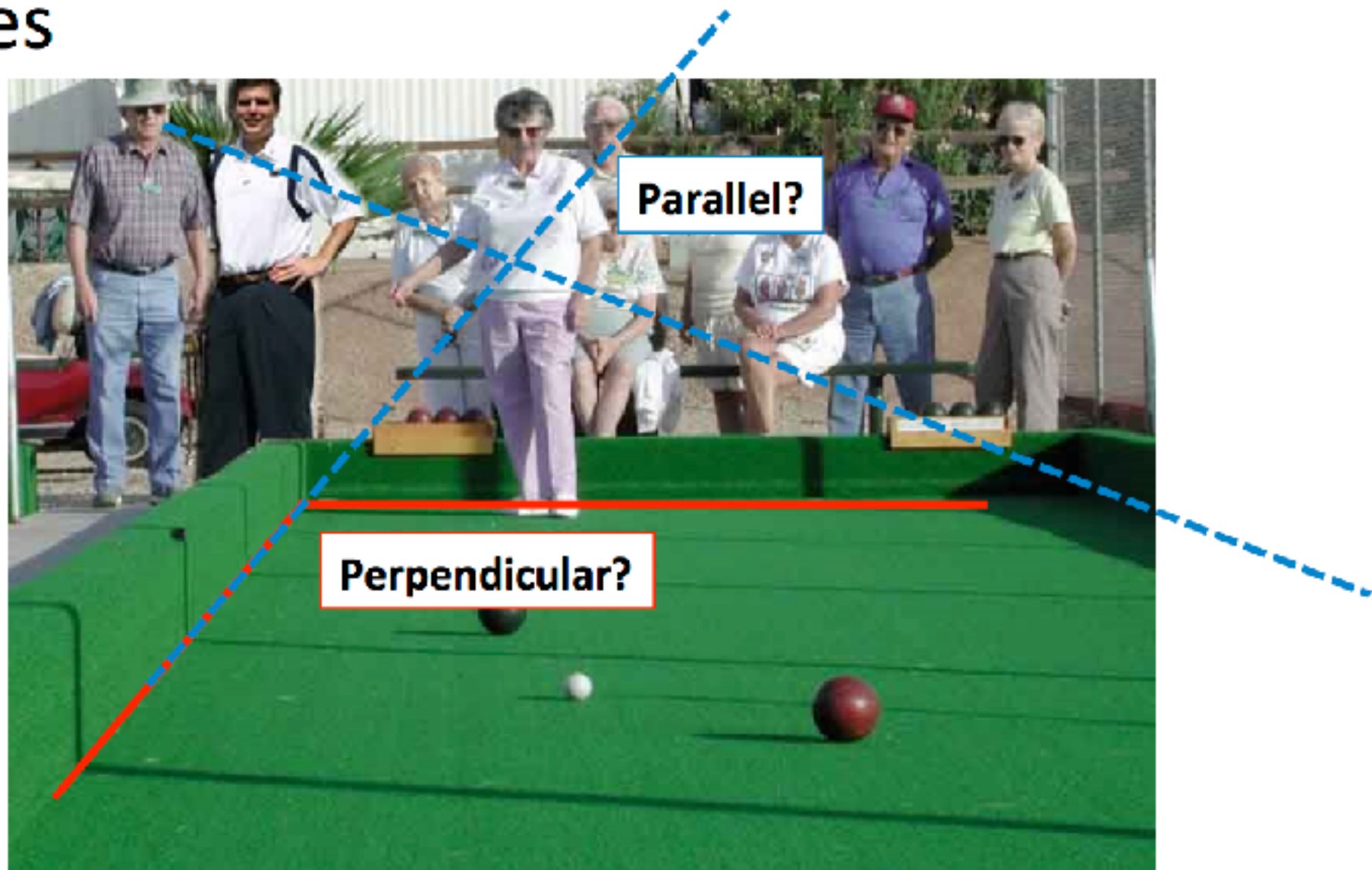


- Object size decreases with distance to the pinhole
- There, given a single projection, if we know the size of the object we can know how far it is.
- But for objects of unknown size, the 3D information seems to be lost.

Projective Geometry

What is lost?

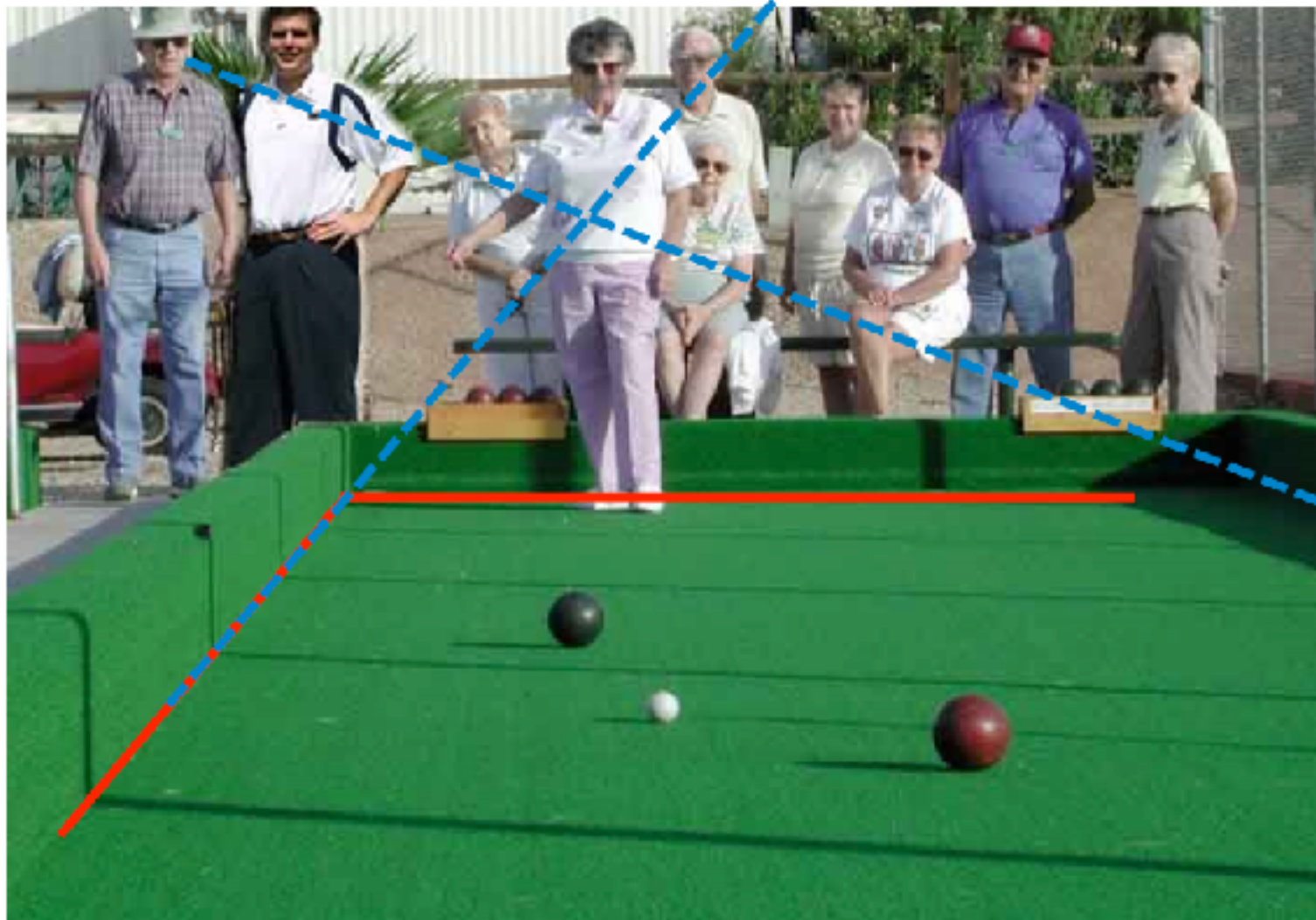
- Length
- Angles



Projective Geometry

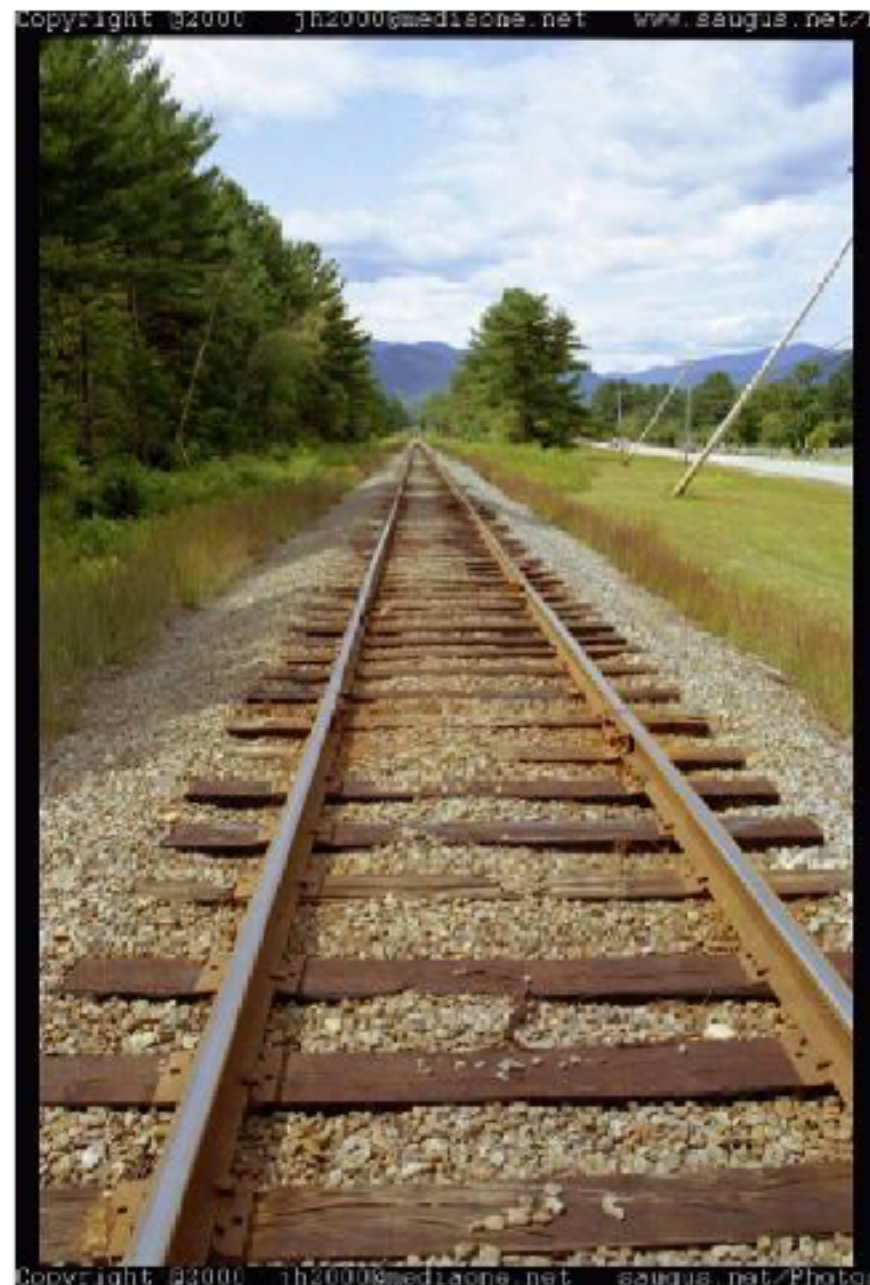
What is preserved?

- Straight lines are still straight

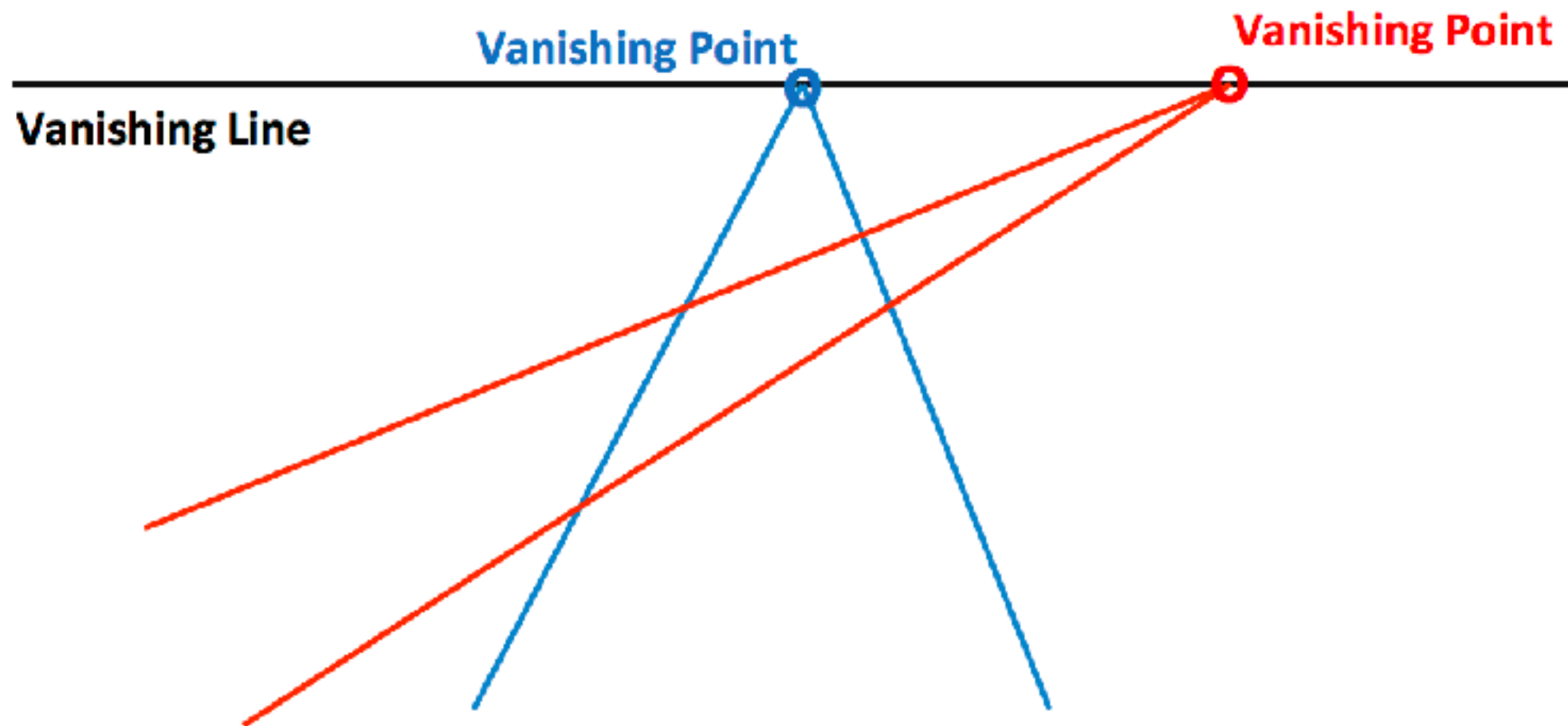


Vanishing points and lines

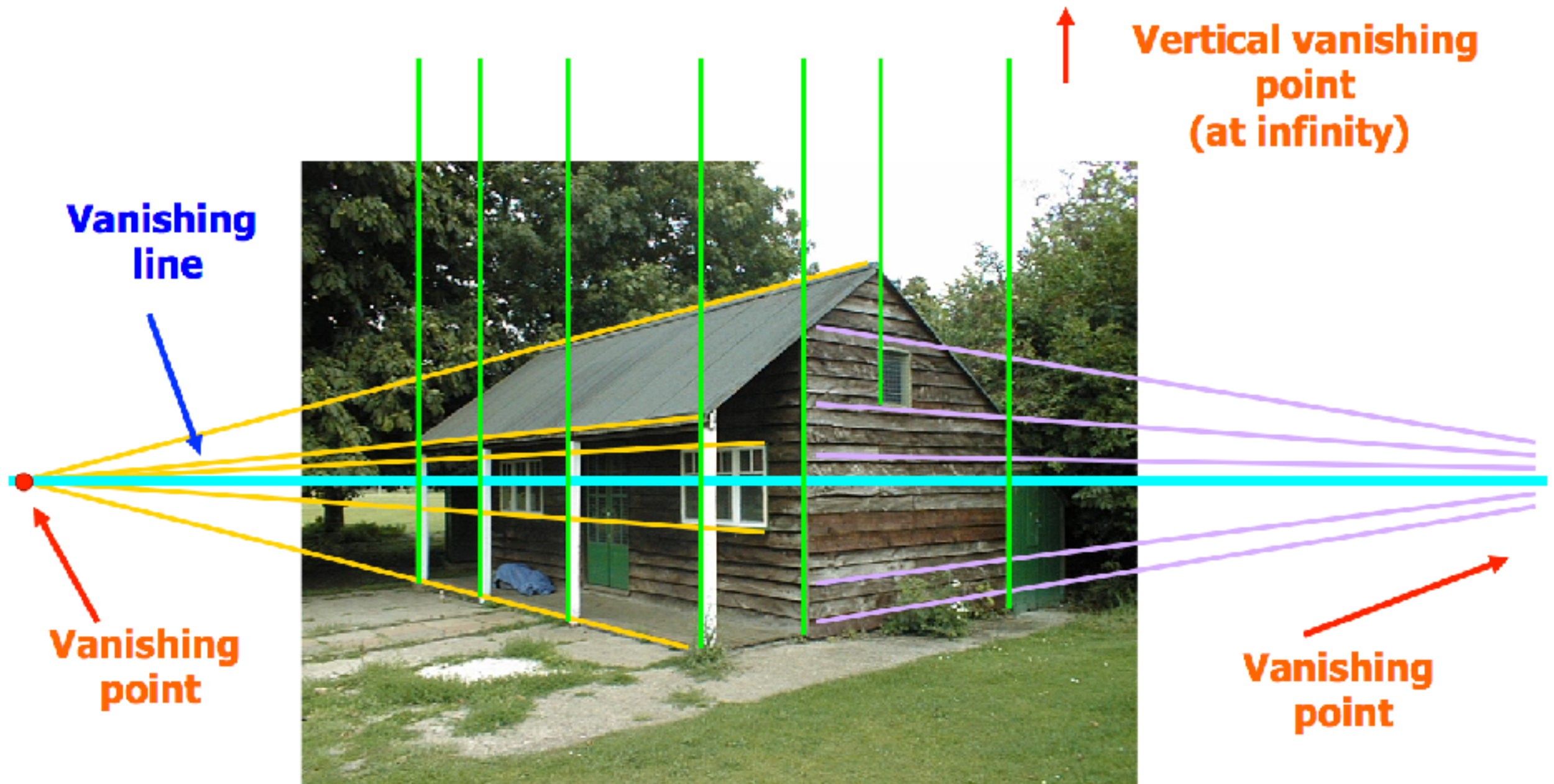
Parallel lines in the world intersect in the image at a “vanishing point”



Vanishing points and lines



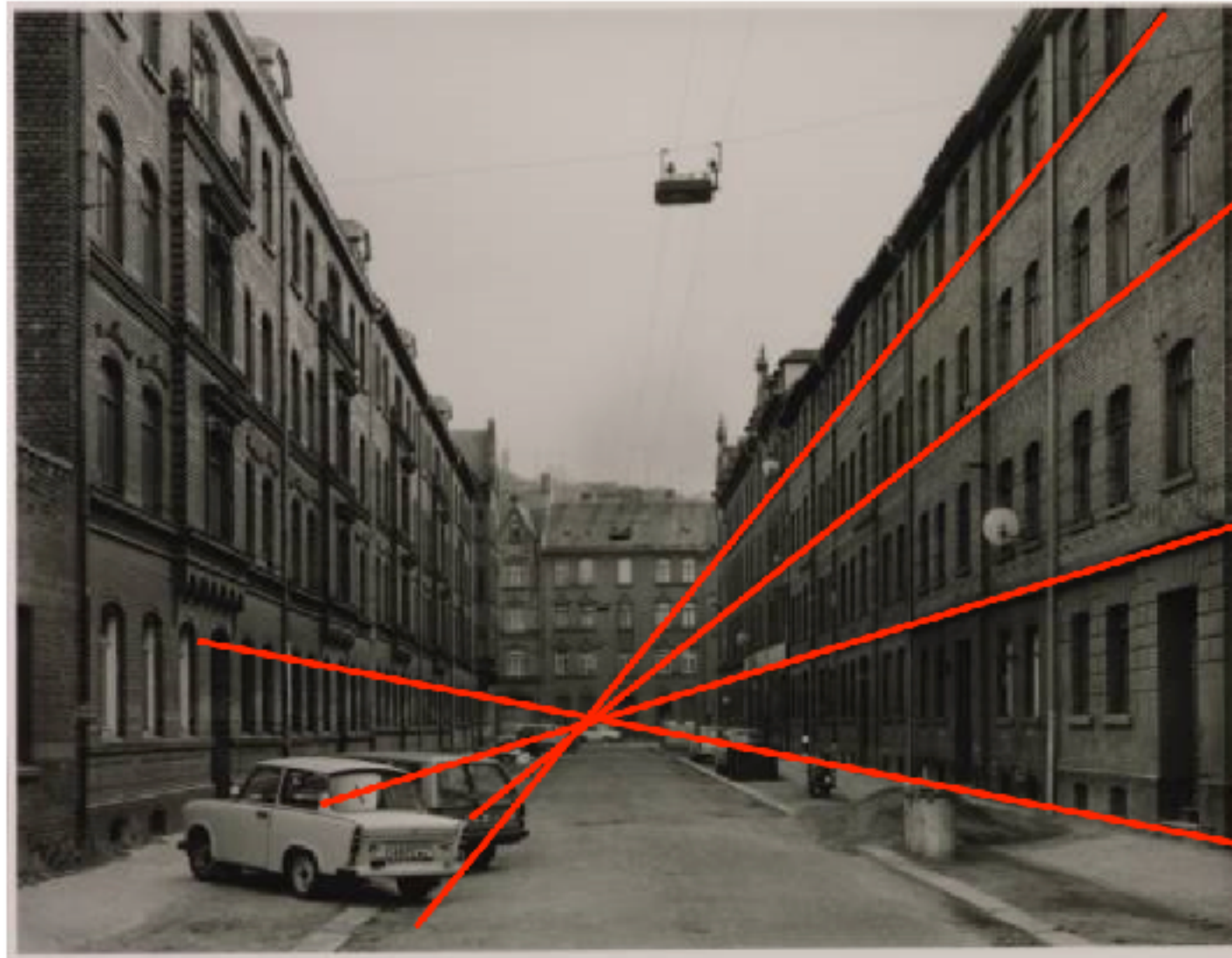
Vanishing points and lines



Vanishing points and lines

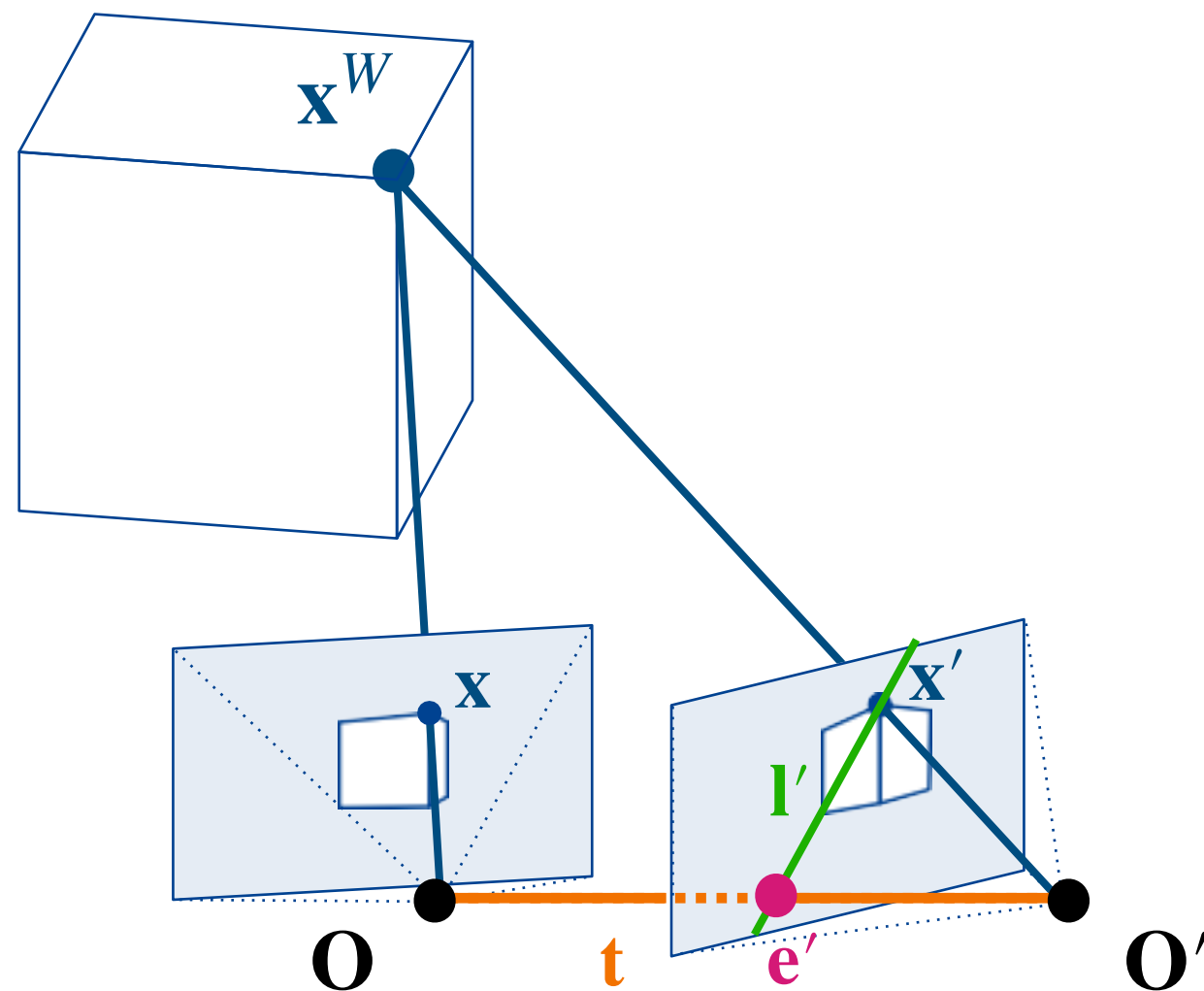


Photo from online Tate collection



Epipolar Geometry (two cameras observing a point)

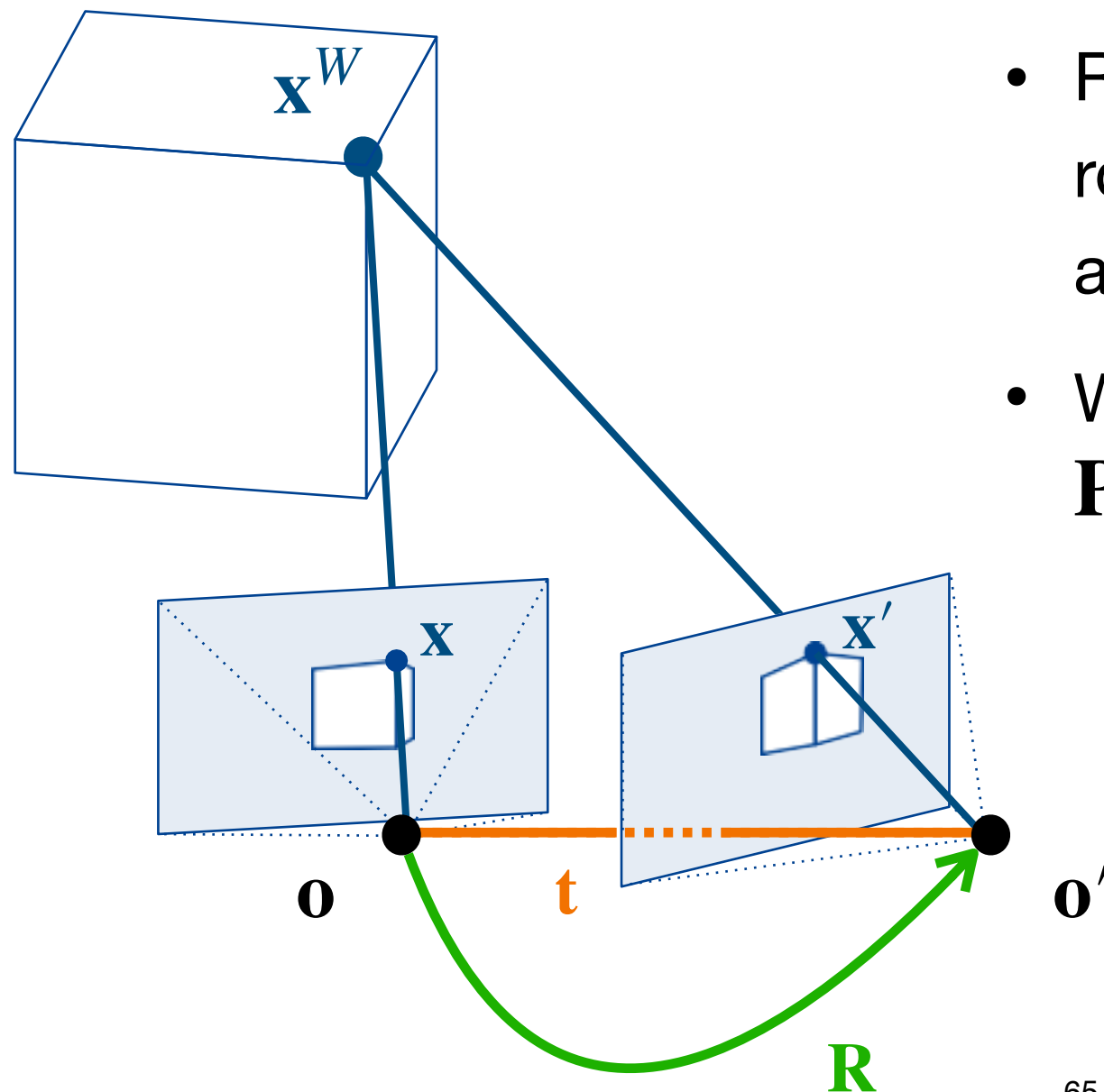
Epipolar geometry

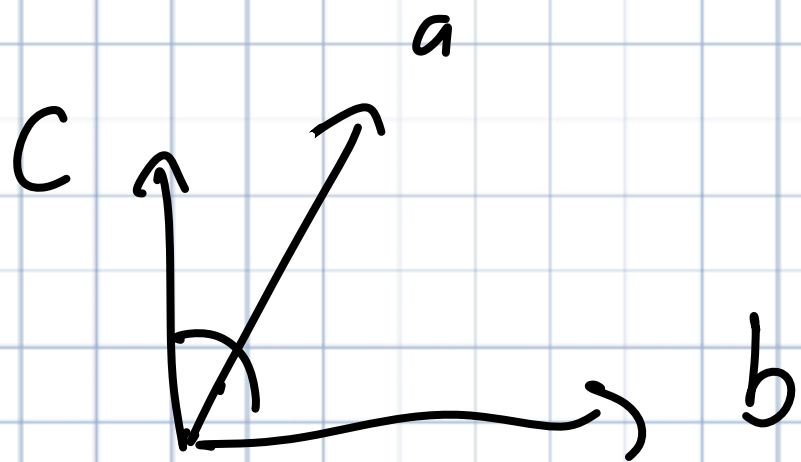


- O, O' : projection centers
- x, x' : image points of x^W
- t : stereo basis
- e' : right epipole
(Projection of O)
- l' : epipolar line
Connection of e' with x' ; contains all points observed as x in the left view

Simplified coordinate systems

- Canonical Kameras: $\mathbf{K} = \mathbf{K}' = \mathbf{I}$
- World coordinate system in the left camera system
- Right Kamera shifted by $\mathbf{t}$ and rotated by $\mathbf{R}$ - points $\mathbf{x}^W$ will thus appear as $\mathbf{R}^T(\mathbf{x}^W - \mathbf{t})$
- We obtain projection matrices $\mathbf{P} = [\mathbf{I} \quad \mathbf{0}]$ und $\mathbf{P}' = [\mathbf{R}^T \quad -\mathbf{R}^T\mathbf{t}]$



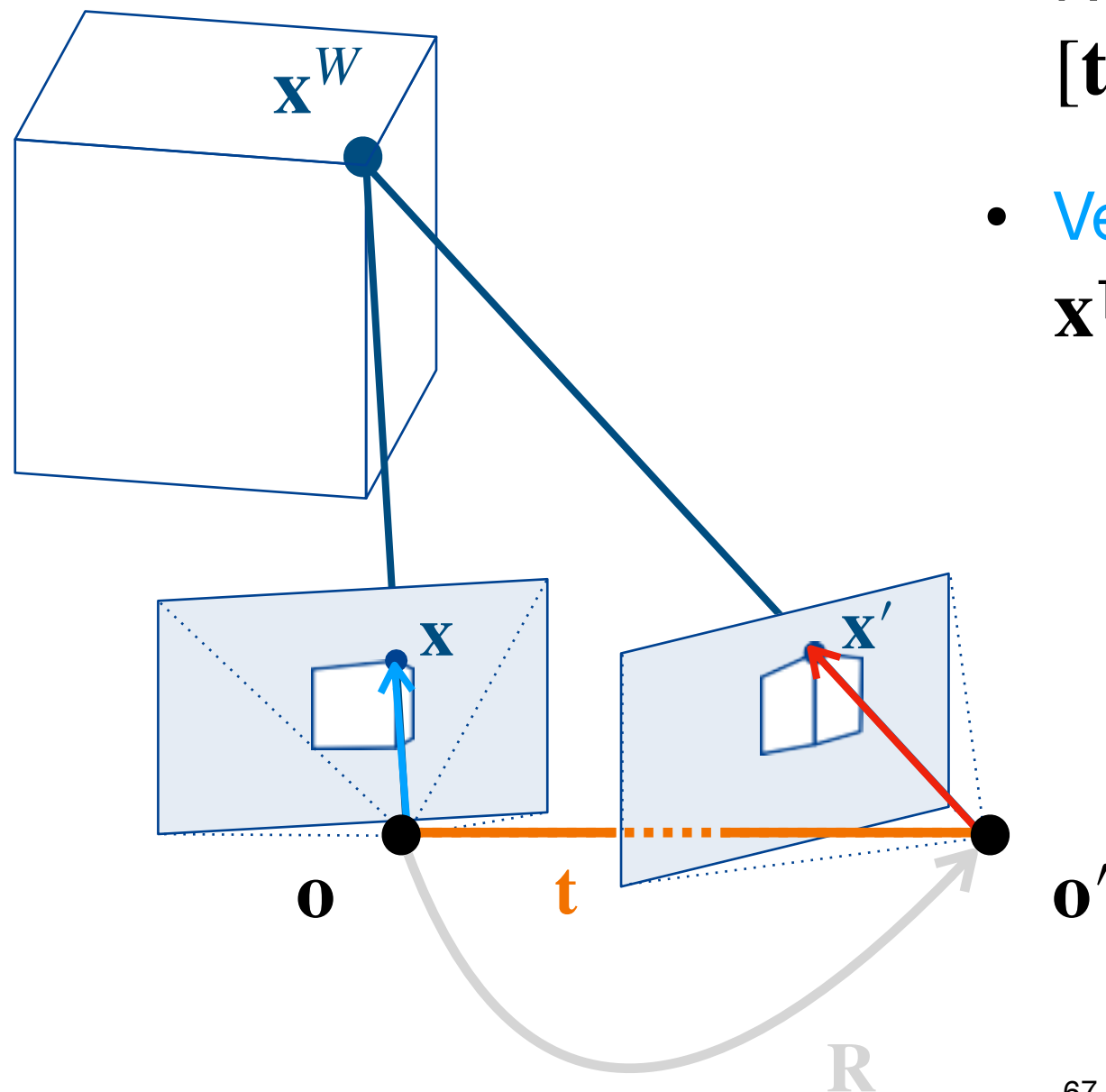


$$c = a \times b = [a]_{\times} \cdot b$$

$$\begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}$$

The essential matrix E

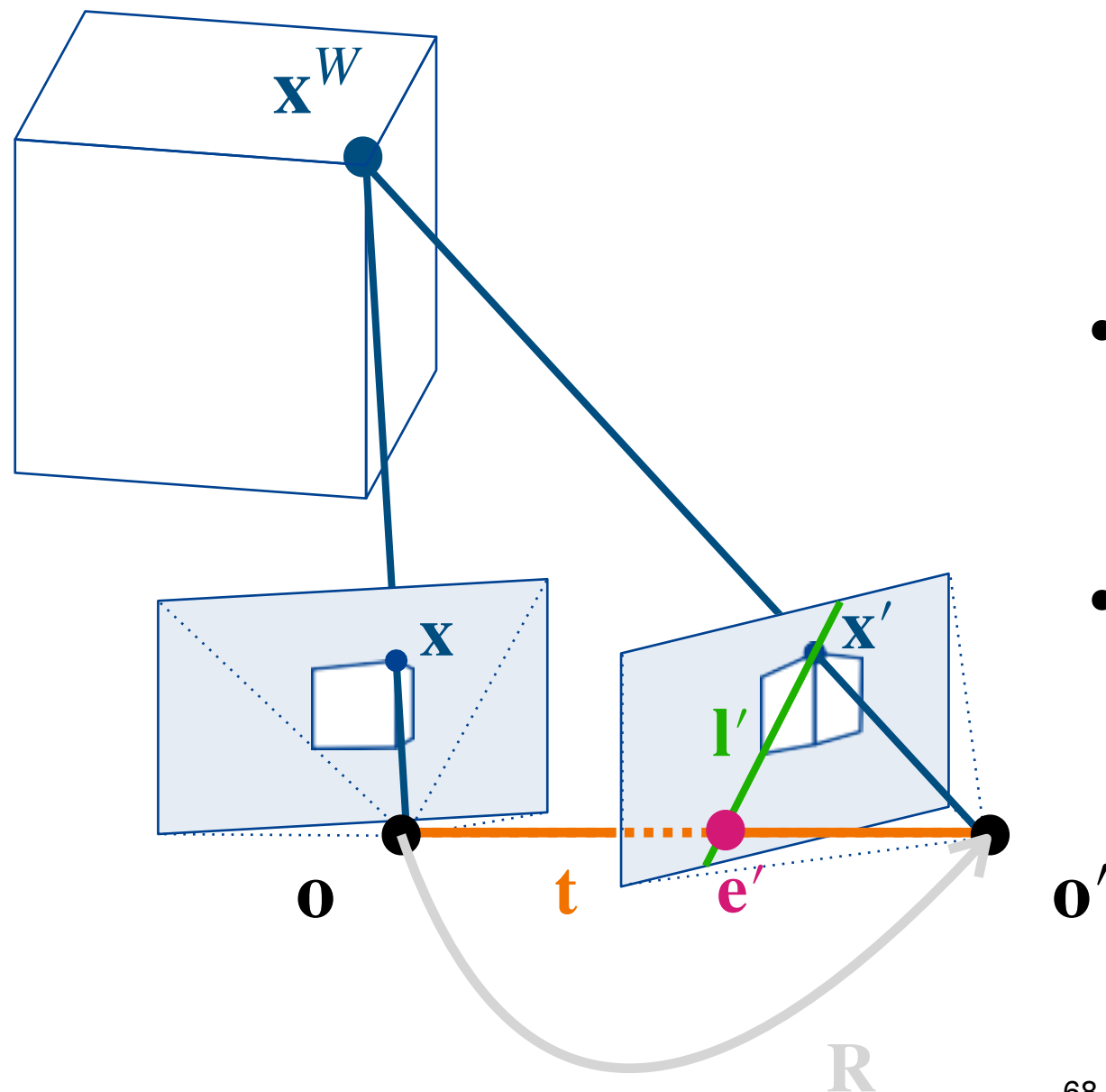
We span the epipolar plane in the coordinate system of the left camera.



- The **red vector** is $\mathbf{R} \mathbf{x}'$
- Normal of the epipolar plane:
 $[\mathbf{t}]_{\times} (\mathbf{R} \mathbf{x}')$
- **Vector $\mathbf{x}$** lies in the plane, hence
 $\mathbf{x}^T [\mathbf{t}]_{\times} \mathbf{R} \mathbf{x}' = 0$

Essential matrix
 $\mathbf{E} = [\mathbf{t}_{\times}] \mathbf{R}$
with epipolar condition
 $\mathbf{x}^T \mathbf{E} \mathbf{x}' = 0$

Properties of $\mathbf{E}$



- $\mathbf{x}'$ lies on the **epipolar line $\mathbf{l}'$** , hence $\mathbf{l}'^T \mathbf{x}' = 0$
- With $\mathbf{x}^T \mathbf{E} \mathbf{x}' = 0$ we see that $\mathbf{l}' = (\mathbf{x}^T \mathbf{E})^T = \mathbf{E}^T \mathbf{x}$ and $\mathbf{l} = \mathbf{E} \mathbf{x}'$
- In particular, $\mathbf{E} \mathbf{e}' = \mathbf{0}$ and $\mathbf{E}^T \mathbf{e} = \mathbf{0}$
- Due to $\mathbf{t}^T \mathbf{E} = \mathbf{t}^T [\mathbf{t}_\times] \mathbf{R} = 0$ we have $\mathbf{t}$ as the null space!

The fundamental matrix $\mathbf{F}$

- Non-canonical pixel coordinates $\mathbf{p}, \mathbf{p}'$ can be converted using the intrinsic parameters to get $\mathbf{x} = \mathbf{K}^{-1}\mathbf{p}$ and $\mathbf{x}' = \mathbf{K}'^{-1}\mathbf{p}'$
- Then we have $\mathbf{x}^\top \mathbf{E} \mathbf{x}' = (\mathbf{K}^{-1}\mathbf{p})^\top \mathbf{E} \mathbf{K}'^{-1}\mathbf{p}' = \mathbf{p}^\top \mathbf{K}^{-\top} \mathbf{E} \mathbf{K}'^{-1} \mathbf{p}'$
- Thus there is a matrix $\mathbf{F}$ with $\mathbf{p}^\top \mathbf{F} \mathbf{p}' = 0$

Fundamental Matrix

$$\mathbf{F} = \mathbf{K}^{-\top} [\mathbf{t}_\times] \mathbf{R} \mathbf{K}'^{-1}$$

$$\mathbf{p}^\top \mathbf{F} \mathbf{p}' = 0$$